

**Klasifikasi Transaksi Penipuan Kartu Kredit
Skala Besar Menggunakan Pendekatan
Neighbor Sampling GraphSAGE**

Proposal Tugas Akhir

(IF-47-DSIS.03)

103012300116

Muhammad Hendika Putra



Program Studi Sarjana Informatika

Fakultas Informatika

Universitas Telkom

Bandung

2026

Lembar Persetujuan

Klasifikasi Transaksi Penipuan Kartu Kredit Skala Besar
Menggunakan Pendekatan Neighbor Sampling GraphSAGE

*Large-Scale Credit Card Fraud Transaction Classification Using
GraphSAGE Neighbor Sampling*

NIM: 103012300116
Muhammad Hendika Putra

Proposal ini diajukan sebagai usulan pembuatan tugas akhir pada
Program Studi Sarjana Informatika
Fakultas Informatika Universitas Telkom

Bandung, 25 Mei 2026
Menyetujui

Calon Pembimbing 1

Dr. Fitriyani, S.Si., M.T.
NIP: 10830001

Abstrak

Penipuan kartu kredit merupakan permasalahan penting dalam ekosistem keuangan digital karena dapat menimbulkan kerugian finansial besar dan membutuhkan sistem deteksi yang akurat, efisien, serta mampu bekerja pada data transaksi berskala besar. Pendekatan berbasis aturan dan *machine learning* konvensional masih memiliki keterbatasan karena umumnya menganalisis transaksi secara tabular dan terisolasi, sehingga kurang mampu menangkap pola relasional antar entitas seperti pengguna, transaksi, dan *merchant*. Tantangan ini semakin kompleks karena data penipuan memiliki karakteristik *extreme class imbalance*, pola penipuan yang langka, serta kemungkinan adanya perilaku kamufase dalam bentuk *heterophily*.

Penelitian ini bertujuan untuk menganalisis pengaruh strategi *neighbor sampling* pada GraphSAGE terhadap performa klasifikasi transaksi penipuan kartu kredit berskala besar. Metode yang digunakan adalah membangun representasi graf bipartit yang menghubungkan node pengguna, transaksi, dan *merchant*, kemudian melatih model GraphSAGE menggunakan tiga strategi *sampling*, yaitu *Uniform Random Sampling* sebagai *baseline*, *Topology-Aware Sampling*, dan *Importance-Based Sampling*. Ketiga strategi dibandingkan dengan arsitektur GraphSAGE yang sama agar perbedaan performa dapat dikaitkan dengan mekanisme *sampling*. Evaluasi dilakukan menggunakan F1-Score, Recall, AUPRC, dan *inference time*.

Dataset yang digunakan adalah *IBM Synthetic Credit Card Transactions Dataset* yang memuat 24.386.900 transaksi sintetis dengan 15 fitur transaksi dan rasio penipuan sekitar 0,12%. Hipotesis awal penelitian ini adalah bahwa strategi *neighbor sampling* yang mempertimbangkan topologi jaringan dan tingkat kepentingan node mampu meningkatkan kemampuan GraphSAGE dalam menangkap sinyal penipuan langka dibandingkan *Uniform Random Sampling*. Dengan demikian, *Topology-Aware Sampling* dan *Importance-Based Sampling* diharapkan dapat meningkatkan Recall, F1-Score, dan AUPRC, dengan tetap menjaga *trade-off* terhadap waktu inferensi.

Kata Kunci: deteksi penipuan, kartu kredit, graphsage, neighbor sampling, graph neural network, class imbalance.

Daftar Isi

Abstrak	i
Daftar Isi	ii
I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Perumusan Masalah	3
1.3 Batasan Masalah	4
1.4 Tujuan	4
1.5 Hipotesis	5
1.6 Rencana Kegiatan	6
1.6.1 Studi Literatur	6
1.6.2 Pengumpulan dan Persiapan Data	6
1.6.3 Analisis dan Perancangan Sistem	7
1.6.4 Implementasi Sistem	7
1.6.5 Analisis Hasil Implementasi	7
1.6.6 Penulisan Laporan	8
1.7 Jadwal Kegiatan	8
II Kajian Pustaka	9
2.1 Penelitian Terdahulu	9
2.2 Fraud dalam Sistem Keuangan	12
2.3 Deteksi Penipuan Kartu Kredit	13
2.4 Representasi Graf Transaksi Keuangan	13
2.5 Graph Neural Networks dan Arsitektur GraphSAGE	14
2.6 Mekanisme <i>Neighbor Sampling</i> pada GraphSAGE	16
2.7 <i>Heterophily</i> dan Kamufase Penipu	17
2.8 Penanganan Ketidakseimbangan Kelas pada Graf	18
2.9 Dataset Berskala Besar untuk Deteksi Penipuan	19
2.10 Metrik Evaluasi	20
2.10.1 <i>Accuracy</i>	20
2.10.2 <i>Precision</i>	20
2.10.3 <i>Recall</i>	20

2.10.4	F1- <i>Score</i>	20
2.10.5	ROC-AUC	21
2.10.6	AUPRC	21
III	Alur Pemodelan	22
3.1	Gambaran Umum Alur Pemodelan	22
3.2	Pengumpulan dan Pra-pemrosesan Data	22
3.3	Konstruksi Graf Bipartit Transaksi	25
3.4	Arsitektur Model GraphSAGE	26
3.5	Perancangan Strategi <i>Neighbor Sampling</i>	30
3.5.1	<i>Uniform Random Sampling</i> (Baseline)	30
3.5.2	<i>Topology-Aware Sampling</i>	30
3.5.3	<i>Importance-Based Sampling</i>	31
3.6	Prosedur Pelatihan Model	33
3.7	Skenario Pengujian dan Evaluasi	35
	Daftar Pustaka	39
	Lampiran	42

Bab I

Pendahuluan

Bab ini memaparkan landasan awal penelitian yang mencakup latar belakang permasalahan, perumusan masalah, batasan masalah, tujuan penelitian, hipotesis, rencana kegiatan, serta jadwal pelaksanaan. Secara keseluruhan, bab ini bertujuan untuk memberikan gambaran menyeluruh mengenai urgensi, ruang lingkup, dan arah penelitian tentang klasifikasi transaksi penipuan kartu kredit berskala besar menggunakan pendekatan *neighbor sampling* GraphSAGE.

1.1 Latar Belakang

Penipuan kartu kredit merupakan salah satu ancaman terbesar dalam ekosistem keuangan digital modern. Menurut laporan Nilson Report, kerugian akibat penipuan kartu kredit secara global diproyeksikan mencapai 397,40 miliar dolar selama satu dekade ke depan [1]. Di Amerika Serikat saja, Komisi Perdagangan Federal (FTC) mencatat kerugian konsumen akibat penipuan mencapai hampir 8,8 miliar dolar pada tahun 2022, meningkat lebih dari 30% dibandingkan tahun sebelumnya [2]. Besarnya dampak finansial ini mendorong kebutuhan mendesak akan sistem deteksi penipuan yang akurat, efisien, dan mampu beroperasi secara *real-time* pada skala jutaan transaksi per hari.

Selama bertahun-tahun, sistem deteksi penipuan kartu kredit mengandalkan pendekatan berbasis aturan (*rule-based*) dan algoritme *machine learning* konvensional yang memproses data secara tabular. Metode-metode ini, seperti *Logistic Regression*, *Random Forest*, *Support Vector Machine* (SVM), dan *XGBoost*, bekerja dengan menganalisis fitur-fitur individual sebuah transaksi seperti nominal, waktu, dan lokasi secara terisolasi [3]. Meskipun memberikan hasil yang cukup baik pada data terstruktur sederhana, pendekatan tabular tersebut memiliki keterbatasan fundamental karena tidak mampu menangkap pola relasional antar entitas transaksi [4]. Padahal, penipuan finansial modern jarang terjadi secara terisolasi. Para pelaku umumnya bergerak dalam sindikat yang terorganisir, menggunakan identitas yang saling terkait, dan melakukan pengujian kartu di *merchant* yang sama dalam rentang waktu berdekatan [5].

Untuk mengatasi keterbatasan tersebut, pendekatan berbasis graf (*graph-*

based) telah muncul sebagai solusi yang menjanjikan. Transaksi keuangan dapat direpresentasikan sebagai sebuah jaringan di mana pengguna, transaksi, dan *merchant* menjadi simpul (*node*), sementara interaksi di antara mereka menjadi sisi (*edge*) [6]. Dengan representasi ini, model tidak hanya menganalisis sebuah transaksi secara individual, tetapi juga mampu melacak jejak strukturalnya dalam jaringan dan mengenali pola *guilt by association*, yaitu kondisi di mana sebuah transaksi yang terhubung dengan entitas atau *merchant* yang memiliki riwayat mencurigakan akan mendapatkan tingkat risiko yang lebih tinggi [7].

Graph Neural Networks (GNN) telah terbukti menjadi alat yang sangat efektif untuk mengeksplorasi struktur relasional ini dalam deteksi penipuan [4]. Namun, GNN generasi awal seperti *Graph Convolutional Network* (GCN) bersifat *transduktif*, yang mengharuskan seluruh struktur graf dimuat ke dalam memori komputer sekaligus untuk melakukan kalkulasi. Pendekatan ini jelas tidak dapat diterapkan pada data transaksi perbankan yang skalanya mencapai jutaan *node* dan terus bertambah setiap detiknya [8]. Sebagai solusi atas keterbatasan skalabilitas tersebut, *GraphSAGE* (*Graph Sample and Aggregate*) [9] diperkenalkan sebagai kerangka kerja GNN yang bersifat *induktif*. GraphSAGE tidak memerlukan seluruh graf saat pelatihan, melainkan mempelajari fungsi agregasi yang dapat langsung menghasilkan representasi numerik (*embedding*) untuk transaksi baru tanpa perlu melatih ulang seluruh model dari awal.

Meskipun GraphSAGE telah terbukti mampu menangani masalah skalabilitas [10], algoritme ini memiliki titik lemah inheren pada mekanisme pengambilan sampel tetangganya (*neighbor sampling*). Secara bawaan, GraphSAGE menggunakan *Uniform Random Sampling*, yaitu pengambilan sampel tetangga secara acak dan merata. Pada domain deteksi penipuan kartu kredit, pendekatan ini menimbulkan dua permasalahan serius. Pertama, data penipuan memiliki ketidakseimbangan kelas yang ekstrem (*extreme class imbalance*), di mana jumlah transaksi normal dapat jauh melebihi transaksi penipuan [11]. Kondisi ini menyebabkan sinyal penipuan yang kecil akan “terencerkan” (*representation dilution*) atau hilang tertutup oleh data normal selama proses agregasi, berujung pada tingginya angka *False Negative* [12]. Kedua, para penipu modern sering kali berkamufase dengan sengaja bertransaksi bersama entitas atau *merchant* yang berstatus normal untuk menyembunyikan jejak mereka, sebuah fenomena yang dikenal sebagai *heterophily* [13]. Ketika sistem hanya mengambil sampel tetangga secara acak, ia justru lebih banyak memilih tetangga normal dari penipu yang berkamufase, sehingga informasi krusial yang mengidentifikasi aktivitas penipuan menjadi terabaikan [5].

Berbagai penelitian terdahulu telah berupaya mengatasi permasalahan *heterophily* dan *class imbalance* ini melalui berbagai pendekatan. *CARE-GNN* [5] menggunakan *reinforcement learning* untuk memilih jumlah tetangga yang

optimal. *PC-GNN* [11] memperkenalkan *label-balanced sampler* untuk mengatasi ketidakseimbangan kelas. *H2-FDetector* [13] membedakan agregasi untuk koneksi homophilic dan heterophilic. *BalancerGNN* [12] mengusulkan *balanced neighbor sampling* untuk dataset tidak seimbang. Meskipun pendekatan-pendekatan ini menunjukkan peningkatan performa, sebagian besar berfokus pada modifikasi fungsi *loss*, arsitektur agregasi, atau teknik *data augmentation* sehingga komponen *neighbor sampling* pada GraphSAGE itu sendiri belum dioptimalkan secara langsung dan eksplisit. Akibatnya, mekanisme *sampling* yang seharusnya menjadi titik intervensi paling mendasar dalam rantai pemrosesan graf masih beroperasi dengan pendekatan seragam yang tidak mempertimbangkan aspek topologi jaringan, tingkat kepentingan node (*node centrality*), maupun kondisi ketidakseimbangan kelas secara simultan.

Penelitian ini mengusulkan analisis dan optimasi mekanisme *neighbor sampling* pada GraphSAGE untuk meningkatkan performa klasifikasi transaksi penipuan kartu kredit. Eksperimen dilakukan pada *IBM Synthetic Credit Card Transactions Dataset* yang memuat lebih dari 24 juta transaksi sintetis dengan rasio penipuan hanya sekitar 0,12%, menjadikannya representasi nyata dari kondisi *extreme class imbalance* dan skala besar yang menjadi fokus penelitian ini [14]. Dataset ini menyediakan 15 fitur tabular per transaksi, termasuk nominal transaksi (*Amount*), waktu, metode pembayaran (*Use Chip*), kode kategori *merchant* (MCC), serta lokasi geografis *merchant*, yang seluruhnya akan dimanfaatkan sebagai fitur node dalam konstruksi graf bipartit pengguna–transaksi–*merchant*. Dengan memaksa model memilih tetangga secara cerdas berdasarkan bobot topologi jaringan atau tingkat kepentingan sentralitas node, diharapkan sinyal penipuan yang langka dapat tertangkap secara lebih efektif, ketahanan terhadap taktik kamufase penipu meningkat, dan keseimbangan optimal antara akurasi deteksi dengan efisiensi komputasi pada skala besar dapat tercapai.

1.2 Perumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana pengaruh strategi *neighbor sampling* yang dioptimalkan (*topology-aware sampling* dan *importance-based sampling*) terhadap kemampuan GraphSAGE dalam mendeteksi transaksi penipuan kartu kredit yang terkamufase dibandingkan dengan *uniform random sampling* bawaan pada kondisi *extreme class imbalance* dan *heterophily* yang tinggi?
2. Seberapa efektif modifikasi strategi *neighbor sampling* pada GraphSAGE dalam menangkap pola penipuan yang langka (*long-tail fraud*) dan menekan fenomena *oversmoothing* serta *representation dilution* pada graf transaksi berskala besar?

3. Bagaimana *trade-off* antara peningkatan performa deteksi penipuan yang diukur menggunakan metrik *Recall*, *F1-Score*, dan AUPRC terhadap efisiensi komputasi yang dihasilkan oleh masing-masing strategi *neighbor sampling* pada graf yang mengandung jutaan *node* dengan kondisi ketidakseimbangan kelas yang ekstrem?

1.3 Batasan Masalah

Agar penelitian lebih terarah dan tidak melebar ke permasalahan yang di luar lingkup yang ditetapkan, penelitian ini dibatasi pada hal-hal berikut.

1. Penelitian berfokus pada tugas klasifikasi node (transaksi) menggunakan kerangka kerja GraphSAGE dengan representasi graf bipartit yang menghubungkan entitas pengguna, transaksi, dan *merchant*.
2. Dataset yang digunakan adalah *IBM Synthetic Credit Card Transactions Dataset* [14] yang memuat lebih dari 24 juta transaksi dengan 15 fitur tabular per transaksi dan rasio kasus penipuan sebesar $\pm 0,12\%$.
3. Fitur-fitur tabular yang digunakan mencakup nominal transaksi, waktu, metode pembayaran (*Use Chip*), kode kategori *merchant* (MCC), dan informasi lokasi geografis, yang seluruhnya dikonversi menjadi fitur node dalam konstruksi graf.
4. Strategi *neighbor sampling* yang dibandingkan terbatas pada tiga pendekatan, yaitu *uniform random sampling* sebagai *baseline*, *topology-aware sampling* berbasis struktur jaringan, dan *importance-based sampling* berbasis sentralitas node.
5. Evaluasi performa dilakukan menggunakan metrik *F1-Score*, *Recall*, dan *Area Under the Precision-Recall Curve* (AUPRC) untuk mengukur kualitas deteksi, serta *inference time* sebagai ukuran efisiensi komputasi.
6. Penelitian tidak mencakup modifikasi terhadap fungsi *loss*, arsitektur lapisan agregasi, maupun teknik *data augmentation* pada graf.

1.4 Tujuan

Berdasarkan rumusan masalah yang telah ditetapkan, tujuan dari penelitian ini adalah sebagai berikut.

1. Untuk menganalisis dan membandingkan pengaruh strategi *neighbor sampling* yang dioptimalkan (*topology-aware sampling* dan *importance-based sampling*) terhadap kemampuan GraphSAGE dalam mendeteksi transaksi penipuan kartu kredit yang terkamuflase dibandingkan dengan *uniform random sampling* bawaan pada kondisi *extreme class imbalance* dan *heterophily* yang tinggi.

2. Untuk mengevaluasi efektivitas modifikasi strategi *neighbor sampling* pada GraphSAGE dalam menangkap pola penipuan yang langka (*long-tail fraud*) dan menekan fenomena *oversmoothing* serta *representation dilution* pada graf transaksi berskala besar.
3. Untuk mengevaluasi *trade-off* antara peningkatan performa deteksi penipuan dan efisiensi komputasi yang dihasilkan oleh masing-masing strategi *neighbor sampling* pada graf yang mengandung jutaan *node* dengan kondisi ketidakseimbangan kelas yang ekstrem.

1.5 Hipotesis

Penelitian ini mengajukan hipotesis bahwa modifikasi mekanisme *neighbor sampling* pada GraphSAGE dari pendekatan *uniform random sampling* menjadi strategi berbasis topologi jaringan (*topology-aware*) atau berbasis tingkat kepentingan node (*importance-based*) akan secara signifikan meningkatkan performa deteksi penipuan kartu kredit, khususnya pada kondisi *extreme class imbalance* dan *heterophily* yang tinggi.

Argumen yang mendasari hipotesis ini bertumpu pada karakteristik fundamental dari *uniform random sampling* yang memperlakukan semua tetangga secara setara tanpa mempertimbangkan relevansi topologis atau tingkat kepentingan mereka dalam konteks deteksi penipuan. Pada graf transaksi dengan kondisi *heterophily* tinggi, di mana penipu secara sengaja terhubung dengan mayoritas entitas normal, pengambilan sampel acak justru akan mendominasi agregasi dengan informasi dari tetangga normal sehingga sinyal penipuan terabaikan [15]. Sebaliknya, pendekatan *topology-aware sampling* yang memprioritaskan tetangga berdasarkan bobot struktural jaringan seperti nilai sentralitas, kekuatan koneksi antar node, atau derajat homofili lokal diyakini mampu mempertahankan dan memperkuat sinyal penipuan yang langka selama proses agregasi [12].

Perbedaan mendasar dengan pendekatan terdahulu terletak pada fokus modifikasi. Penelitian ini secara spesifik merekayasa lapisan *sampling* pada GraphSAGE sebagai titik intervensi utama, alih-alih memodifikasi fungsi *loss*, arsitektur agregasi, atau melakukan *data augmentation* sebagaimana yang dilakukan pada sebagian besar penelitian sebelumnya [11, 12]. Dengan demikian, modifikasi yang diusulkan tetap kompatibel dengan paradigma induktif GraphSAGE yang memungkinkan inferensi *real-time* pada transaksi baru tanpa pelatihan ulang.

Adapun hasil yang diperkirakan dari penelitian ini adalah strategi *optimized sampling* akan menghasilkan nilai *Recall* dan AUPRC yang lebih tinggi dibandingkan *uniform random sampling* karena kemampuannya yang lebih baik dalam menangkap sinyal penipuan minoritas. Di antara kedua strategi yang diusulkan, *topology-aware sampling* diperkirakan menunjukkan keunggulan da-

lam mendeteksi penipu yang berkamuflase di lingkungan *merchant* normal dibandingkan *importance-based sampling*. Sementara itu, strategi *sampling* yang lebih cerdas diperkirakan membutuhkan waktu inferensi yang sedikit lebih tinggi dibandingkan *uniform random sampling* akibat *overhead* kalkulasi bobot, meskipun diharapkan tetap dalam batas kelayakan untuk sistem *real-time*.

Dalam penelitian ini, peningkatan performa deteksi penipuan diukur melalui perbandingan nilai *F1-Score*, *Recall*, dan *Area Under the Precision-Recall Curve* (AUPRC) antara model GraphSAGE dengan *uniform random sampling* sebagai *baseline* dan model GraphSAGE dengan strategi *neighbor sampling* yang dioptimalkan. Suatu strategi *sampling* dianggap memberikan peningkatan performa apabila mampu menghasilkan nilai *Recall*, *F1-Score*, atau AUPRC yang lebih tinggi secara konsisten pada data pengujian tanpa menyebabkan penurunan efisiensi komputasi yang tidak proporsional. Pemilihan ketiga metrik tersebut didasarkan pada karakteristik deteksi penipuan yang memiliki distribusi kelas sangat tidak seimbang (*extreme class imbalance*), sehingga metrik seperti akurasi tidak cukup representatif dalam menggambarkan kemampuan model mendeteksi transaksi penipuan.

1.6 Rencana Kegiatan

Penelitian ini akan dilaksanakan melalui serangkaian tahapan yang sistematis dan terstruktur, sebagaimana diuraikan pada subbab-subbab berikut.

1.6.1 Studi Literatur

Tahap ini mencakup pengkajian mendalam terhadap literatur yang relevan, meliputi konsep dasar dan perkembangan GNN untuk deteksi penipuan finansial, arsitektur dan mekanisme kerja GraphSAGE beserta berbagai varian *neighbor sampling*-nya, serta permasalahan *class imbalance*, *heterophily*, dan *oversmoothing* pada graf penipuan. Selain itu, dikaji pula metrik evaluasi yang sesuai untuk kondisi ketidakseimbangan kelas dan dataset publik yang relevan untuk eksperimen. Studi literatur dilakukan secara berkelanjutan sepanjang pengerjaan penelitian.

1.6.2 Pengumpulan dan Persiapan Data

Tahap ini meliputi pengunduhan dan pemrosesan awal *IBM Synthetic Credit Card Transactions Dataset* [14] yang tersedia secara publik melalui platform Kaggle. Dataset ini memuat lebih dari 24 juta baris transaksi sintetis dengan 15 kolom fitur, di antaranya *User* dan *Card* sebagai identifier entitas, *Amount* sebagai nominal transaksi, *Year/Month/Day/Time* sebagai informasi temporal, *Use Chip* sebagai metode pembayaran, *Merchant Name*, *Merchant City*, *Merchant State*, *Zip*, *MCC* sebagai kode kategori *merchant*, *Errors?*, dan label target *Is Fraud?*. Rasio kasus penipuan pada dataset ini hanya sekitar

0,12% dari total transaksi, yang merepresentasikan kondisi *extreme class imbalance* secara nyata.

Proses persiapan data mencakup konstruksi graf bipartit pengguna–transaksi–*merchant* dengan kolom *User* dan *Merchant Name* sebagai node penghubung, serta rekayasa fitur node transaksi dari kolom tabular yang tersedia. Rekayasa fitur ini meliputi normalisasi *Amount*, ekstraksi fitur temporal seperti *hour of day* dan *day of week*, encoding kategorikal untuk *Use Chip* dan *MCC*, serta encoding lokasi geografis *merchant*. Kolom identifier murni seperti *User* dan *Card* dihapus dari fitur untuk mencegah kebocoran data (*data leakage*), dan distribusi kelas pada label *Is Fraud?* diverifikasi secara menyeluruh.

1.6.3 Analisis dan Perancangan Sistem

Tahap ini mencakup analisis mendalam terhadap mekanisme *uniform random sampling* pada GraphSAGE dan identifikasi titik-titik kelemahannya pada kondisi *class imbalance* dan *heterophily*. Berdasarkan analisis tersebut, dilakukan perancangan dan formulasi strategi *topology-aware sampling* yang memprioritaskan tetangga berdasarkan bobot struktural jaringan, serta strategi *importance-based sampling* yang memprioritaskan tetangga berdasarkan nilai sentralitas node. Tahap ini juga mencakup perancangan protokol eksperimen komparatif yang komprehensif untuk memastikan validitas perbandingan antar strategi.

1.6.4 Implementasi Sistem

Tahap ini meliputi implementasi seluruh komponen sistem menggunakan *framework* Python dengan pustaka seperti PyTorch Geometric atau DGL (*Deep Graph Library*). Implementasi mencakup konstruksi dan *preprocessing* graf transaksi, implementasi *baseline* GraphSAGE dengan *uniform random sampling*, implementasi modifikasi *topology-aware sampling*, implementasi modifikasi *importance-based sampling*, serta implementasi *pipeline* evaluasi dan pencatatan metrik secara otomatis.

1.6.5 Analisis Hasil Implementasi

Tahap ini mencakup pelaksanaan eksperimen komparatif secara sistematis dan analisis mendalam terhadap hasil yang diperoleh. Analisis dilakukan terhadap perbandingan performa antar strategi *sampling* menggunakan metrik *F1-Score*, *Recall*, AUPRC, dan *inference time*, diikuti dengan kajian kasus di mana masing-masing strategi berhasil atau gagal mendeteksi penipuan. Selain itu, dikaji pula pengaruh skala graf dan tingkat ketidakseimbangan kelas terhadap efisiensi komputasi, serta *trade-off* antara akurasi deteksi dan latensi inferensi pada tiap strategi.

1.6.6 Penulisan Laporan

Tahap ini mencakup dokumentasi seluruh proses dan hasil penelitian ke dalam laporan Tugas Akhir yang terstruktur dan komprehensif, mulai dari latar belakang, kajian pustaka, metodologi, hasil eksperimen, analisis, hingga kesimpulan dan saran.

1.7 Jadwal Kegiatan

Jadwal pelaksanaan penelitian ini direncanakan berlangsung selama 6 bulan, sebagaimana ditampilkan pada Tabel 1.1.

Tabel 1.1: Jadwal kegiatan proposal tugas akhir

No	Kegiatan	Bulan ke-																									
		1				2				3				4				5				6					
1	Studi Literatur																										
2	Pengumpulan dan Persiapan Data																										
3	Analisis dan Perancangan Sistem																										
4	Implementasi Sistem																										
5	Analisis Hasil Implementasi																										
6	Penulisan Laporan																										

Bab II

Kajian Pustaka

Bab ini menyajikan kajian terhadap literatur yang relevan dengan penelitian tentang klasifikasi transaksi penipuan kartu kredit menggunakan pendekatan *neighbor sampling* pada GraphSAGE. Kajian dimulai dari tinjauan penelitian terdahulu untuk memetakan posisi penelitian ini di antara karya-karya terkait, kemudian dilanjutkan dengan fondasi teoritis yang mencakup representasi graf, arsitektur GNN dan GraphSAGE, mekanisme *neighbor sampling*, tantangan *heterophily* dan *class imbalance*, hingga dataset berskala besar yang digunakan.

2.1 Penelitian Terdahulu

Deteksi penipuan berbasis graf telah berkembang dari pendekatan representasi jaringan menuju model *Graph Neural Networks* yang mampu mempelajari pola relasional antar entitas. Tinjauan sistematis oleh Motie dan Raahemi [4] menunjukkan bahwa sebagian besar penelitian berfokus pada modifikasi fungsi *loss*, arsitektur agregasi, teknik augmentasi, dan strategi pemrosesan graf. Meskipun demikian, mekanisme *neighbor sampling* pada GraphSAGE masih menjadi area yang belum banyak dibahas sebagai titik intervensi utama, padahal mekanisme ini menentukan tetangga mana yang masuk ke proses agregasi dan secara langsung memengaruhi kualitas representasi simpul pada graf yang tidak seimbang.

Penelitian terdahulu yang dipilih pada Tabel 2.1 merupakan referensi yang paling relevan dengan arah proposal ini. Pemilihan dilakukan dengan mempertimbangkan keterkaitan terhadap GraphSAGE, *neighbor sampling*, deteksi penipuan berbasis graf, *heterophily*, ketidakseimbangan kelas, dan kebutuhan skalabilitas pada graf transaksi berukuran besar.

Tabel 2.1: Ringkasan penelitian terdahulu yang paling relevan

Pustaka	Dataset	Pendekatan	Kontribusi Utama	Keterbatasan
Hamilton et al. 2017 [9]	Citation, Reddit, dan PPI	GraphSAGE dengan mekanisme <i>sample and aggregate</i>	Menjadi fondasi pembelajaran representasi graf induktif dengan <i>neighbor sampling</i> untuk menghasilkan <i>embedding</i> pada simpul baru	Tidak dirancang khusus untuk deteksi penipuan, <i>heterophily</i> , dan ketidakseimbangan kelas ekstrem
Zeng et al. 2020 [8]	PPI, Reddit, dan graf berskala besar lain	GraphSAINT dengan <i>graph sampling</i>	Menunjukkan bahwa strategi <i>sampling</i> dapat meningkatkan efisiensi pelatihan GNN pada graf besar dan mengurangi masalah <i>neighbor explosion</i>	Berfokus pada <i>graph-level sampling</i> , bukan optimasi <i>neighbor sampling</i> pada GraphSAGE untuk deteksi penipuan
Dou et al. 2020 [5]	YelpChi dan Amazon	CARE-GNN dengan seleksi tetangga berbasis kemiripan dan <i>reinforcement learning</i>	Menjelaskan <i>feature camouflage</i> dan <i>relation camouflage</i> serta menunjukkan pentingnya memilih tetangga yang informatif pada graf penipuan	Dataset tidak berupa transaksi kartu kredit berskala jutaan simpul dan pendekatan RL berpotensi menambah beban komputasi
Liu et al. 2021 [11]	YelpChi, Amazon, dan T-Finance	PC-GNN dengan <i>label-balanced sampler</i> dan <i>neighborhood sampler</i>	Menangani ketidakseimbangan kelas dengan konstruksi <i>mini-batch</i> yang lebih seimbang dan penyaringan tetangga yang lebih informatif	Pemilihan tetangga belum secara khusus membandingkan strategi berbasis topologi jaringan dan sentralitas simpul
Van Belle et al. 2022 [6]	Dataset kartu kredit nyata	CATCHM dengan representasi jaringan dan <i>node representation learning</i>	Menunjukkan bahwa transaksi kartu kredit dapat direpresentasikan sebagai jaringan untuk menangkap hubungan tidak langsung antar entitas	Pendekatan utama tidak berfokus pada GraphSAGE dan tidak mengevaluasi optimasi <i>neighbor sampling</i>

Lanjutan Tabel 2.1

Pustaka	Dataset	Pendekatan	Kontribusi Utama	Keterbatasan
Van Belle et al. 2022 [10]	Dataset kartu kredit nyata yang sangat tidak seimbang	GraphSAGE induktif dengan <i>graph-level undersampling</i>	Membuktikan relevansi GraphSAGE untuk deteksi penipuan kartu kredit dan menunjukkan manfaat pembelajaran induktif pada data transaksi yang terus bertambah	<i>Neighbor sampling</i> masih bersifat seragam dan belum diarahkan untuk menangkap pola kamufase penipu
Huang et al. 2022 [16]	DGraph	Dataset graf keuangan berskala besar	Menyediakan dataset graf finansial besar dengan jutaan simpul dan menunjukkan perbedaan struktur antara simpul normal dan anomali	Dataset tidak secara khusus berbentuk transaksi kartu kredit dan tidak menyediakan evaluasi strategi <i>neighbor sampling</i> GraphSAGE
Shi et al. 2022 [13]	YelpChi dan Amazon	H2-FDetector dengan pemisahan koneksi <i>homophilic</i> dan <i>heterophilic</i>	Menunjukkan bahwa koneksi <i>heterophilic</i> dapat menjadi sinyal diskriminatif untuk mendeteksi penipu yang berkamufase	Fokus utama berada pada desain agregasi, sedangkan mekanisme pemilihan tetangga tidak menjadi objek evaluasi utama
Wu et al. 2023 [17]	YelpChi, Amazon, dan DGraph	SplitGNN dengan pemisahan sinyal spektral	Membuktikan bahwa <i>heterophily</i> pada penipu berkaitan dengan pergeseran sinyal graf dan perlu ditangani secara berbeda dari simpul normal	Pendekatan spektral tidak secara langsung menjawab pemilihan tetangga pada GraphSAGE induktif
Tong dan Shen 2023 [18]	YelpChi, Amazon, dan Elliptic	HHLN-GNN dengan <i>subgraph generator</i> , <i>neighborhood sampler</i> , dan <i>self-attention</i>	Menggabungkan penanganan ketidakseimbangan kelas dengan pemanfaatan koneksi homogen dan heterogen pada graf penipuan	Evaluasi <i>trade-off</i> antara kualitas deteksi dan latensi pada graf transaksi jutaan simpul belum menjadi fokus utama
Boyapati et al. 2024 [12]	Medicare, Equifax, IEEE, dan asuransi kendaraan	BalancerGNN dengan <i>balanced neighbor sampling</i>	Menunjukkan bahwa penyeimbangan tetangga pada tahap <i>sampling</i> dapat meningkatkan sensitivitas model terhadap kelas minoritas	Tidak diuji secara khusus pada dataset kartu kredit berskala besar dengan rasio penipuan sangat rendah

Berdasarkan Tabel 2.1, terlihat bahwa penelitian terdahulu telah membahas deteksi penipuan dari berbagai sisi, mulai dari representasi graf, arsitektur GNN, seleksi tetangga, penanganan *heterophily*, hingga penanganan ketidakseimbangan kelas. Namun, masih terdapat celah penelitian pada evaluasi eksplisit terhadap *uniform random sampling* GraphSAGE dibandingkan dengan *optimized sampling* yang memanfaatkan informasi topologi dan tingkat kepentingan simpul. Celah ini menjadi dasar penelitian yang diusulkan karena deteksi penipuan kartu kredit berskala besar membutuhkan model yang tidak hanya akurat terhadap kelas minoritas, tetapi juga efisien ketika diterapkan pada graf berukuran jutaan simpul.

2.2 Fraud dalam Sistem Keuangan

Fraud atau penipuan merupakan tindakan memperoleh keuntungan secara tidak sah melalui manipulasi, penyalahgunaan informasi, pemalsuan identitas, maupun bentuk kecurangan lainnya yang merugikan pihak lain [4]. Dalam sektor keuangan, fraud menjadi salah satu ancaman utama karena dapat menyebabkan kerugian finansial yang besar bagi individu, lembaga keuangan, maupun penyedia layanan pembayaran. Seiring meningkatnya digitalisasi layanan keuangan, pola fraud juga berkembang menjadi semakin kompleks dan sering kali melibatkan hubungan antar entitas yang sulit diidentifikasi melalui analisis transaksi secara individual [4, 5].

Secara umum, fraud pada sistem keuangan dapat diklasifikasikan menjadi beberapa kategori. Pertama, *credit card fraud*, yaitu penggunaan kartu kredit atau informasi kartu kredit tanpa otorisasi pemilik yang sah. Bentuk fraud ini dapat berupa pencurian kartu fisik, pencurian data kartu, maupun *card-not-present fraud* yang terjadi pada transaksi daring [3]. Kedua, *identity theft*, yaitu penggunaan identitas individu lain untuk memperoleh akses terhadap layanan keuangan atau melakukan transaksi ilegal. Ketiga, *account takeover*, yaitu pengambilalihan akun pengguna secara tidak sah sehingga pelaku dapat melakukan transaksi atas nama pemilik akun. Keempat, *merchant fraud*, yaitu tindakan kecurangan yang dilakukan oleh pihak *merchant*, misalnya manipulasi transaksi, pengajuan tagihan fiktif, atau penyalahgunaan informasi pembayaran pelanggan [4].

Penelitian ini berfokus pada *credit card fraud detection*, yaitu identifikasi transaksi kartu kredit yang dikategorikan sebagai fraud atau non-fraud berdasarkan informasi transaksi dan hubungan antar entitas dalam graf. Pada dataset IBM Synthetic Credit Card Transactions yang digunakan dalam penelitian ini, label fraud hanya tersedia pada level transaksi (*transaction-level fraud label*), sedangkan entitas pengguna (*user*) dan *merchant* tidak memiliki label fraud maupun non-fraud. Oleh karena itu, pengguna dan *merchant* dimanfaatkan sebagai simpul relasional untuk membantu pembentukan struktur graf dan mendukung proses pembelajaran representasi transaksi oleh model

GraphSAGE [14].

2.3 Deteksi Penipuan Kartu Kredit

Penipuan kartu kredit merujuk pada penggunaan tidak sah dana dalam sebuah transaksi yang umumnya dilakukan melalui kartu kredit maupun kartu debit [3]. Dari perspektif keilmuan data, permasalahan ini merupakan tugas klasifikasi biner di mana setiap transaksi harus diklasifikasikan sebagai sah atau curang berdasarkan fitur yang tersedia. Kerugian akibat penipuan kartu kredit secara global diproyeksikan mencapai 397,40 miliar dolar selama satu dekade ke depan [1], sementara di Amerika Serikat saja kerugian konsumen mencapai hampir 8,8 miliar dolar pada tahun 2022, meningkat lebih dari 30% dibandingkan tahun sebelumnya [2].

Pendekatan berbasis aturan dan *machine learning* konvensional seperti *Logistic Regression*, *Random Forest*, *Support Vector Machine*, dan *XGBoost* bekerja dengan menganalisis fitur-fitur individual sebuah transaksi secara terisolasi [3]. Meskipun memberikan hasil yang memadai pada data terstruktur sederhana, pendekatan tabular ini tidak mampu menangkap pola relasional yang terjalin di antara entitas-entitas transaksi [4]. Dou et al. [5] mendokumentasikan bahwa para penipu umumnya bergerak dalam sindikat yang terorganisir, menggunakan identitas yang saling terkait, dan melakukan pengujian kartu di *merchant* yang sama dalam rentang waktu berdekatan. Pola perilaku sindikat semacam ini hanya dapat terdeteksi apabila analisis dilakukan pada level relasi antar entitas dalam jaringan transaksi yang lebih luas.

Dalam kondisi *class imbalance* yang ekstrem, metrik evaluasi standar seperti akurasi tidak dapat diandalkan karena model yang selalu memprediksi kelas mayoritas pun akan menghasilkan akurasi yang tampak tinggi [4]. *Area Under the Precision-Recall Curve* (AUPRC) merupakan metrik yang lebih representatif karena mengukur kemampuan model mendeteksi kelas minoritas pada berbagai ambang keputusan tanpa terpengaruh oleh jumlah *true negative* yang sangat besar [4]. *Recall* secara langsung mencerminkan kemampuan model menekan *False Negative*, sementara *F1-Score* menyeimbangkan *Precision* dan *Recall* sehingga memberikan gambaran holistik tentang kualitas deteksi [3].

2.4 Representasi Graf Transaksi Keuangan

Pendekatan berbasis graf menggeser cara pandang analisis transaksi dari perspektif individual menuju perspektif relasional. Secara formal, sebuah graf dinotasikan sebagai $G = (V, E, X)$ di mana V adalah himpunan simpul (*node*), $E \subseteq V \times V$ adalah himpunan sisi (*edge*), dan $X \in \mathbb{R}^{|V| \times d}$ adalah matriks fitur simpul berdimensi d [9]. Dalam konteks transaksi keuangan, entitas seperti pengguna, transaksi, dan *merchant* dimodelkan sebagai simpul, sementara interaksi di antara mereka dimodelkan sebagai sisi.

Van Belle et al. [6] memperkenalkan desain jaringan tripartit yang memasukkan *node* buatan untuk secara langsung menginkorporasikan informasi penipuan, yang memungkinkan model menangkap hubungan tidak langsung antara pengguna dan *merchant*. Xiang et al. [7] membangun graf temporal berdasarkan kesamaan atribut transaksi sehingga sisi yang terbentuk merepresentasikan kedekatan perilaku antar transaksi, bukan hanya hubungan langsung antar entitas. Zou et al. [1] menganalisis distribusi homofili *node* pada tiga dataset penipuan yang berbeda dan menemukan bahwa *node* penipu secara konsisten berada di sekitar nilai homofili mendekati nol, yang menjadi bukti empiris kuantitatif dari perilaku kamufase mereka di dalam jaringan graf. Huang et al. [16] menunjukkan bahwa dalam dataset DGraph, simpul anomali dan simpul normal memiliki perbedaan yang signifikan dalam struktur, distribusi tetangga, dan dinamika temporal.

2.5 Graph Neural Networks dan Arsitektur GraphSAGE

Graph Neural Networks (GNN) adalah keluarga model *deep learning* yang dirancang untuk mempelajari representasi pada data berbentuk graf. Dalam konteks deteksi penipuan finansial, GNN relevan karena mampu memanfaatkan struktur jaringan transaksi untuk menangkap pola relasional antar entitas, seperti pengguna, transaksi, akun, dan *merchant*, yang sulit ditangkap oleh pendekatan tabular konvensional [4]. Secara umum, GNN bekerja melalui mekanisme *message passing*, yaitu proses pembaruan representasi simpul dengan mengagregasi informasi dari simpul-simpul tetangganya.

Secara formal, untuk lapisan ke- k , representasi simpul v dapat diperbarui sebagai berikut:

$$h_v^{(k)} = \text{UPDATE}^{(k)} \left(h_v^{(k-1)}, \text{AGGREGATE}^{(k)} \left(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\} \right) \right) \quad (2.1)$$

di mana $\mathcal{N}(v)$ adalah himpunan tetangga simpul v , $h_v^{(k)}$ adalah representasi simpul v pada lapisan ke- k , dan $h_v^{(0)} = x_v$ merupakan fitur awal simpul. Mekanisme ini memungkinkan model mempelajari representasi simpul berdasarkan fitur intrinsik simpul dan informasi struktural dari lingkungan lokalnya.

GraphSAGE atau *Graph Sample and Aggregate* merupakan salah satu arsitektur GNN yang diperkenalkan oleh Hamilton et al. [9]. Berbeda dari pendekatan transduktif yang mempelajari *embedding* individual untuk setiap simpul pada graf tetap, GraphSAGE mempelajari fungsi agregasi yang dapat digunakan untuk menghasilkan representasi bagi simpul baru. Oleh karena itu, GraphSAGE bersifat induktif dan sesuai untuk graf yang terus berkembang, seperti graf transaksi keuangan yang selalu menerima transaksi baru [9, 10].

Arsitektur umum GraphSAGE terdiri atas tiga tahap utama, yaitu *sampling*, *aggregation*, dan *prediction*. Pertama, GraphSAGE mengambil sejumlah tetangga dari lingkungan simpul target pada setiap lapisan. Kedua, fitur tetangga yang telah disampling diagregasi menggunakan fungsi agregator tertentu. Ketiga, hasil agregasi tersebut digabungkan dengan representasi simpul target untuk membentuk representasi baru yang kemudian dapat digunakan pada tugas prediksi, seperti klasifikasi simpul [9]. Dalam paper aslinya, Hamilton et al. mengusulkan beberapa fungsi agregasi, antara lain *mean aggregator*, LSTM aggregator, dan *pooling aggregator* [9]. Kajian lain pada domain deteksi penipuan kartu kredit juga menjelaskan bahwa GraphSAGE menghasilkan *embedding* melalui fitur simpul yang disampling dan diagregasi dari lingkungan lokal, sehingga dapat digunakan untuk menghasilkan representasi pada simpul yang belum pernah dilihat sebelumnya [6].

Secara matematis, proses agregasi tetangga pada GraphSAGE dapat dituliskan sebagai berikut:

$$h_{\mathcal{N}(v)}^{(k)} = \text{AGGREGATE}^{(k)}(\{h_u^{(k-1)}, u \in \mathcal{N}(v)\}) \quad (2.2)$$

Setelah representasi tetangga diperoleh, GraphSAGE menggabungkan representasi simpul target dari lapisan sebelumnya dengan hasil agregasi tetangga, kemudian mentransformasikannya menggunakan matriks bobot yang dipelajari. Proses tersebut dapat dirumuskan sebagai berikut:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, h_{\mathcal{N}(v)}^{(k)} \right) \right) \quad (2.3)$$

di mana $h_{\mathcal{N}(v)}^{(k)}$ adalah hasil agregasi tetangga pada lapisan ke- k , $W^{(k)}$ adalah matriks bobot yang dipelajari, σ adalah fungsi aktivasi non-linear, dan CONCAT adalah operasi penggabungan antara representasi simpul target dan hasil agregasi tetangga. Setelah melewati K lapisan, GraphSAGE menghasilkan representasi akhir z_v yang dapat digunakan untuk klasifikasi.

Pada GraphSAGE, istilah lapisan (*layer*) merujuk pada satu tahap agregasi *hop* tetangga, bukan pada lapisan jaringan saraf tersembunyi (*hidden layer*) semata. Setiap lapisan memperluas jangkauan informasi yang dapat diakses oleh suatu simpul sejauh satu tingkat ketetanggaan (*hop*). Dengan kata lain, lapisan pertama (*Layer 1*) mengagregasi informasi dari tetangga langsung (*1-hop neighbors*), sedangkan lapisan kedua (*Layer 2*) mengagregasi informasi yang berasal dari tetangga-tetangga tersebut (*2-hop neighbors*) [9]. Semakin banyak lapisan yang digunakan, semakin luas pula konteks struktural yang dapat dimanfaatkan dalam pembentukan representasi simpul.

Sebagai ilustrasi konkret, misalkan sebuah simpul transaksi T terhubung langsung dengan simpul pengguna U (*user*) dan simpul *merchant* M . Selanjutnya, U juga terhubung dengan transaksi-transaksi lain T_2 dan T_3 , sedangkan M terhubung dengan pengguna lain U_2 dan U_3 .

- **Layer 1 (agregasi tetangga 1-hop):** Representasi simpul target T diperbarui dengan mengagregasi informasi langsung dari U dan M yang terhubung langsung dengannya. Pada tahap ini, GraphSAGE melakukan *sampling* sejumlah tetangga langsung lalu mengagregasi fitur-fiturnya.
- **Layer 2 (agregasi tetangga 2-hop):** Representasi T diperkaya lebih lanjut dengan informasi dari entitas yang berjarak dua lompatan, yaitu T_2, T_3 (melalui U) serta U_2, U_3 (melalui M). Informasi dari 2-hop ini terlebih dahulu diagregasi menjadi representasi baru U dan M pada lapisan pertama, baru kemudian digunakan di lapisan kedua untuk memperbarui representasi T .

Dengan demikian, GraphSAGE dengan dua lapisan mampu memanfaatkan informasi hingga lingkungan *2-hop* secara bertahap: lapisan pertama menangkap sinyal lokal langsung, sedangkan lapisan kedua menangkap sinyal kontekstual yang lebih luas. Namun, penggunaan lapisan yang terlalu banyak dapat meningkatkan biaya komputasi dan berpotensi menyebabkan *oversmoothing*, yaitu kondisi ketika representasi antar simpul menjadi semakin mirip sehingga kemampuan model dalam membedakan kelas menurun [8].

Penggunaan *neighbor sampling* pada GraphSAGE bertujuan untuk membatasi jumlah tetangga yang diproses pada setiap lapisan agar pelatihan dapat dilakukan secara efisien pada graf besar. Zeng et al. [8] menjelaskan bahwa GraphSAGE menggunakan *uniform node sampling* pada tetangga di lapisan sebelumnya dan menetapkan batas ukuran sampel untuk mengendalikan kompleksitas komputasi mini-batch. Dengan demikian, GraphSAGE dapat mengurangi permasalahan *neighbor explosion*, yaitu peningkatan jumlah simpul pendukung secara eksponensial ketika kedalaman lapisan bertambah [8].

Dalam konteks penelitian ini, GraphSAGE digunakan sebagai arsitektur dasar karena memiliki kemampuan induktif dan mekanisme *sampling* yang cocok untuk graf transaksi berskala besar. Namun, kualitas representasi yang dihasilkan sangat bergantung pada tetangga yang dipilih selama proses *neighbor sampling*. Hal ini menjadi penting pada deteksi penipuan kartu kredit karena data penipuan bersifat sangat tidak seimbang dan pola relasi penipu dapat bercampur dengan banyak simpul normal. Oleh karena itu, penelitian ini tidak hanya menggunakan GraphSAGE sebagai model dasar, tetapi juga mengevaluasi bagaimana strategi *neighbor sampling* memengaruhi performa deteksi penipuan pada graf transaksi berskala besar.

2.6 Mekanisme *Neighbor Sampling* pada GraphSAGE

Mekanisme *neighbor sampling* merupakan komponen kritis dalam GraphSAGE yang secara langsung menentukan informasi apa yang dapat dijangkau dan diintegrasikan oleh setiap simpul selama agregasi. Dalam implementasi standar, GraphSAGE menggunakan *uniform random sampling* di mana

sejumlah k tetangga dipilih dari lingkungan setiap simpul dengan probabilitas yang sama [9]. Mekanisme ini menimbulkan dua permasalahan serius pada domain deteksi penipuan kartu kredit.

Permasalahan pertama adalah *representation dilution* akibat *extreme class imbalance*. Ketika rasio penipuan hanya sekitar 0,12% seperti pada IBM Synthetic Credit Card Transactions Dataset [14], jumlah tetangga normal yang disampling akan mendominasi secara berlebihan dibandingkan tetangga yang terlibat dalam penipuan. Liu et al. [11] menunjukkan bahwa kondisi ini menyebabkan sinyal penipuan yang sangat langka terencerkan selama proses agregasi, menghasilkan representasi simpul yang bias ke arah karakteristik kelas mayoritas dan berujung pada tingginya angka *False Negative*. Boyapati et al. [12] mengkonfirmasi bahwa ketidakseimbangan kelas berdampak langsung pada cara graf dibangun, yang merupakan langkah kritis dalam banyak arsitektur GNN.

Permasalahan kedua berkaitan dengan *heterophily*. Dou et al. [5] mendokumentasikan bahwa penipu secara strategis bertransaksi dengan mayoritas entitas normal untuk menyamarkan pola penipuan, sehingga ketika tetangga disampling secara acak dari lingkungan simpul penipu, mayoritas tetangga yang terpilih berupa simpul normal dan informasi krusial menjadi terabaikan [12]. Tian et al. [19] mengusulkan pendekatan *adaptive neighbor sampling* berbasis *cosine similarity* dan bobot sisi untuk menggantikan *uniform random sampling*, serta memperkenalkan metrik diversitas berbasis entropi untuk memitigasi *oversmoothing* secara eksplisit, yang terbukti mengungguli seluruh *baseline* pada tiga dataset keuangan. Penelitian ini menempatkan diri secara berbeda dengan secara khusus membandingkan dua strategi *sampling* yang berbeda secara filosofi, yaitu *topology-aware sampling* berbasis bobot struktural jaringan dan *importance-based sampling* berbasis sentralitas *node*, pada skala graf yang jauh lebih besar dari yang diuji dalam penelitian-penelitian sebelumnya.

2.7 *Heterophily* dan Kamufase Penipu

Sebagian besar arsitektur GNN beroperasi berdasarkan asumsi homofili, yaitu keyakinan bahwa simpul yang terhubung cenderung memiliki label yang sama [13]. Dalam konteks deteksi penipuan, asumsi ini tidak berlaku secara konsisten karena penipu secara strategis terhubung dengan mayoritas entitas normal. Dou et al. [5] mendokumentasikan dua jenis kamufase yang terbukti secara empiris pada dataset nyata, yaitu *feature camouflage* di mana penipu memanipulasi fitur transaksi agar menyerupai transaksi normal, dan *relation camouflage* di mana penipu secara sengaja membangun koneksi dengan entitas-entitas normal. Zou et al. [1] secara kuantitatif membuktikan bahwa *node* penipu pada tiga dataset penipuan berbeda secara konsisten berada di sekitar nilai homofili mendekati nol, jauh berbeda dari *node* normal yang memiliki

distribusi homofili yang lebih tinggi.

Berbagai penelitian mengusulkan pendekatan untuk mengatasi *heterophily* dari sisi arsitektur. Shi et al. [13] memisahkan jalur agregasi untuk koneksi *homophilic* dan *heterophilic*, di mana koneksi *homophilic* mempropagasikan informasi serupa sementara koneksi *heterophilic* mempropagasikan informasi perbedaan yang bersifat diskriminatif, dengan *prototype prior* yang memandu identifikasi penipu. Li et al. [20] merancang kerangka dua jalur di mana jalur *disparity augment* memperkuat perbedaan antara simpul target dengan tetangga *heterophilic*-nya, sementara jalur *similarity augment* memperkuat kemiripan dengan tetangga *homophilic*-nya dalam lingkungan yang lebih seimbang distribusi labelnya. Wu et al. [17] menemukan dari domain spektral bahwa *heterophily* pada simpul penipu mengakibatkan distribusi energi spektral bergeser dari frekuensi rendah ke frekuensi tinggi, yang menginspirasi penggunaan *band-pass graph filter* fleksibel untuk menangkap sinyal pada rentang frekuensi yang relevan.

Chen et al. [15] mengusulkan dua strategi *node sampling* yang menggabungkan informasi *node* kuat dengan topologi graf, di mana modul SSAM melakukan *up-sampling* untuk menambahkan *node* berguna ke dalam graf yang jarang, sementara modul SNM melakukan *down-sampling* berdasarkan informasi *node* kuat dan fitur asli. Tian et al. [19] mengusulkan pendekatan yang lebih langsung dengan memodifikasi mekanisme *neighbor sampling* menggunakan metrik diversitas berbasis entropi yang secara eksplisit mengurangi *oversmoothing* sambil tetap menangani kamufase penipu melalui kalkulasi kemiripan adaptif.

2.8 Penanganan Ketidakseimbangan Kelas pada Graf

Extreme class imbalance merupakan karakteristik inheren dari data penipuan yang menantang model berbasis GNN. Ketika rasio kelas minoritas hanya beberapa perseribu dari keseluruhan data, model cenderung bias menuju kelas mayoritas karena fungsi *loss* standar memperlakukan semua sampel secara setara tanpa mempertimbangkan ketidakseimbangan distribusi [11]. Pada setting graf, permasalahan ini diperparah karena struktur ketetanggaan itu sendiri juga mencerminkan ketidakseimbangan tersebut, sehingga simpul minoritas secara rata-rata memiliki lebih sedikit tetangga yang berlabel sama.

Pada level data, Liu et al. [11] menggabungkan *label-balanced sampler* untuk konstruksi subgraf dengan *neighborhood sampler* adaptif, yang memastikan setiap *mini-batch* mengandung proporsi simpul minoritas yang lebih representatif. Boyapati et al. [12] mengusulkan *balanced neighbor sampling* yang menyeimbangkan distribusi kelas pada lapisan *sampling* secara langsung, menghasilkan sensitivitas 72,87% hingga 81,23% di seluruh dataset yang diuji. Tong dan Shen [18] memanfaatkan *subgraph generators* dan *neighborhood samplers* untuk menangani ketidakseimbangan kelas, dengan peningkatan F1-

macro hingga 10%, AUC hingga 12,5%, dan GMean hingga 17,3% pada dataset YelpChi.

Pada level algoritma, Huang et al. [21] mengoptimalkan langsung metrik AUPRC sebagai fungsi objektif pelatihan karena AUPRC tidak bias terhadap distribusi label, memisahkan proses optimasi menjadi pencarian parameter klasifikator berbasis gradien stokastik AUC dan pencarian kebijakan *edge pruning* berbasis *reinforcement learning*. Van Belle et al. [10] menunjukkan manfaat konkret dari *undersampling* tingkat graf untuk pembelajaran representasi pada jaringan tidak seimbang. Dari keseluruhan kajian ini, teridentifikasi bahwa meskipun berbagai pendekatan menunjukkan hasil yang menjanjikan, pengujian pada kondisi *extreme class imbalance* dengan rasio serendah 0,12% pada skala jutaan *node* masih terbatas dalam literatur yang ada.

2.9 Dataset Berskala Besar untuk Deteksi Penipuan

Pemilihan dataset yang tepat sangat krusial dalam penelitian deteksi penipuan karena karakteristik data secara langsung mempengaruhi validitas temuan. DGraph [16] merupakan salah satu dataset graf keuangan berskala besar yang tersedia secara publik, memuat sekitar tiga juta simpul, empat juta sisi dinamis, dan satu juta simpul dengan label *ground-truth* yang disediakan oleh Finvolution Group. Huang et al. [16] menunjukkan bahwa simpul anomali dan simpul normal dalam DGraph memiliki perbedaan yang signifikan dalam struktur, distribusi tetangga, dan dinamika temporal, serta bahwa dua juta *background node* tanpa label juga berkontribusi penting pada performa deteksi.

IBM Synthetic Credit Card Transactions Dataset [14] dipilih sebagai dataset utama penelitian ini karena memuat lebih dari 24 juta catatan transaksi sintetis dengan 15 fitur tabular per transaksi, mencakup identitas pengguna dan kartu (*User*, *Card*), informasi temporal (*Year*, *Month*, *Day*, *Time*), nominal transaksi (*Amount*), metode pembayaran (*Use Chip*), identitas dan lokasi *merchant* (*Merchant Name*, *Merchant City*, *Merchant State*, *Zip*), kode kategori *merchant* (*MCC*), catatan kesalahan (*Errors?*), serta label target (*Is Fraud?*). Rasio kasus penipuan hanya sekitar 0,12% dari total transaksi [14], yang merepresentasikan kondisi *extreme class imbalance* dalam level yang paling ekstrem sehingga temuan penelitian akan memiliki relevansi tinggi terhadap kondisi dunia nyata. Kolom *User* dan *Merchant Name* dimodelkan sebagai simpul penghubung dalam graf bipartit, sementara fitur simpul transaksi diturunkan melalui normalisasi *Amount*, ekstraksi fitur temporal, encoding kategorikal untuk *Use Chip* dan *MCC*, serta encoding lokasi geografis *merchant*.

2.10 Metrik Evaluasi

Evaluasi model pada penelitian deteksi penipuan kartu kredit tidak cukup menggunakan akurasi karena dataset memiliki karakteristik ketidakseimbangan kelas yang ekstrem. Penelitian terdahulu pada deteksi penipuan kartu kredit dan deteksi penipuan berbasis GNN menggunakan metrik seperti *precision*, *recall*, *F1-score*, ROC-AUC, dan AUPRC untuk menilai performa model secara lebih tepat pada kelas minoritas [3, 4, 10]. Selain performa klasifikasi, penelitian ini juga mempertimbangkan efisiensi komputasi karena GraphSAGE dan strategi *neighbor sampling* digunakan pada graf transaksi berskala besar [9, 8].

2.10.1 Accuracy

Accuracy mengukur proporsi prediksi yang benar terhadap seluruh data. Metrik ini tetap digunakan sebagai metrik pendukung, tetapi tidak menjadi metrik utama karena dapat bias pada dataset yang sangat tidak seimbang [3, 4].

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.4)$$

2.10.2 Precision

Precision mengukur proporsi transaksi yang diprediksi sebagai penipuan dan benar-benar merupakan transaksi penipuan. Metrik ini digunakan untuk melihat seberapa besar model menghasilkan prediksi penipuan yang tepat [3, 4].

$$Precision = \frac{TP}{TP + FP} \quad (2.5)$$

2.10.3 Recall

Recall mengukur kemampuan model dalam menemukan transaksi penipuan yang sebenarnya terjadi. Metrik ini menjadi metrik utama dalam penelitian karena tujuan utama deteksi penipuan adalah mengurangi transaksi penipuan yang tidak terdeteksi [3, 4, 10].

$$Recall = \frac{TP}{TP + FN} \quad (2.6)$$

2.10.4 F1-Score

F1-score mengukur keseimbangan antara *precision* dan *recall*. Metrik ini digunakan untuk menilai performa model ketika kedua aspek tersebut perlu dipertimbangkan secara bersamaan pada data tidak seimbang [3, 4].

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (2.7)$$

2.10.5 ROC-AUC

ROC-AUC mengukur kemampuan model dalam membedakan transaksi normal dan transaksi penipuan pada berbagai ambang keputusan. ROC-AUC dihitung berdasarkan kurva ROC yang dibentuk dari nilai *True Positive Rate* dan *False Positive Rate*. Metrik ini digunakan sebagai metrik pendukung untuk melihat kemampuan pemisahan kelas secara umum [3, 4].

$$TPR = \frac{TP}{TP + FN} \quad (2.8)$$

$$FPR = \frac{FP}{FP + TN} \quad (2.9)$$

$$AUC = \int_0^1 TPR(FPR) dFPR \quad (2.10)$$

Pada Persamaan 2.8, *True Positive Rate* menunjukkan proporsi transaksi penipuan yang berhasil terdeteksi. Pada Persamaan 2.9, *False Positive Rate* menunjukkan proporsi transaksi normal yang salah diklasifikasikan sebagai penipuan. Pada Persamaan 2.10, nilai AUC menunjukkan luas area di bawah kurva ROC.

2.10.6 AUPRC

AUPRC mengukur luas area di bawah kurva *precision-recall*. Metrik ini relevan untuk dataset dengan kelas positif yang sangat sedikit karena lebih berfokus pada performa model terhadap kelas penipuan dibandingkan kelas normal [4, 10].

$$AP = \sum_n (R_n - R_{n-1}) P_n \quad (2.11)$$

Pada Persamaan 2.11, P_n menunjukkan nilai *precision* pada titik ke- n , sedangkan R_n menunjukkan nilai *recall* pada titik ke- n .

Bab III

Alur Pemodelan

Bab ini memaparkan alur pemodelan secara menyeluruh mulai dari persiapan data hingga evaluasi komparatif ketiga strategi *neighbor sampling* yang diusulkan. Setiap tahapan dirancang agar dapat direproduksi dan dibandingkan secara adil (*fair comparison*), sehingga perbedaan performa yang terukur benar-benar mencerminkan kontribusi dari mekanisme *sampling* yang menjadi fokus penelitian, bukan dari perbedaan arsitektur atau konfigurasi pelatihan lainnya.

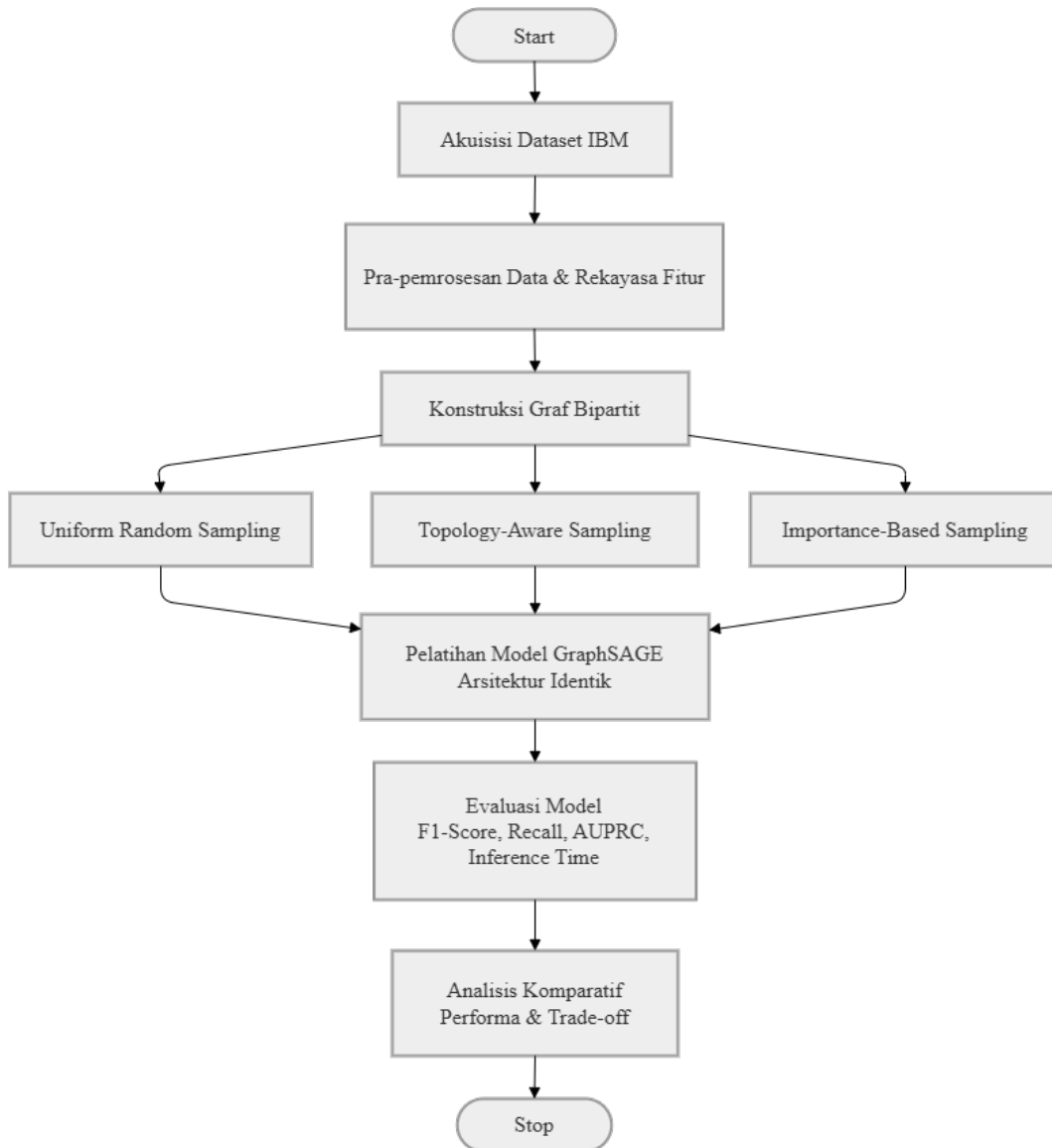
3.1 Gambaran Umum Alur Pemodelan

Penelitian ini mencakup rangkaian tahapan yang saling berurutan, mulai dari akuisisi dan pra-pemrosesan *IBM Synthetic Credit Card Transactions Dataset*, konstruksi graf bipartit yang menghubungkan pengguna, transaksi, dan *merchant*, perancangan strategi *neighbor sampling*, pelatihan model GraphSAGE dengan konfigurasi yang seragam, hingga evaluasi dan analisis komparatif hasil eksperimen. Urutan tersebut dirancang agar setiap tahap memiliki keluaran yang menjadi masukan bagi tahap berikutnya. Gambaran keseluruhan alur penelitian disajikan pada Gambar 3.1.

Gambar 3.1 menampilkan bagan alur penelitian secara ringkas. Proses diawali dari node *Start*, kemudian dilanjutkan dengan akuisisi dataset IBM, pra-pemrosesan data, rekayasa fitur, dan konstruksi graf bipartit yang menghubungkan pengguna, transaksi, dan *merchant*. Setelah graf terbentuk, alur bercabang ke tiga strategi *sampling*, yaitu *Uniform Random Sampling* sebagai *baseline*, *Topology-Aware Sampling*, dan *Importance-Based Sampling*. Seluruh cabang tersebut kembali bertemu pada tahap pelatihan model GraphSAGE dengan konfigurasi arsitektur yang identik. Model yang telah dilatih kemudian dievaluasi menggunakan F1-Score, *Recall*, AUPRC, dan *inference time*. Hasil evaluasi digunakan sebagai dasar analisis komparatif dan analisis *trade-off* antar strategi sebelum proses berakhir pada node *Stop*.

3.2 Pengumpulan dan Pra-pemrosesan Data

Dataset yang digunakan dalam penelitian ini adalah *IBM Synthetic Credit Card Transactions Dataset* [14] yang tersedia secara publik melalui platform



Gambar 3.1: Alur pemodelan penelitian secara keseluruhan.

Kaggle. Dataset ini memuat 24.386.900 baris transaksi sintetis dalam format CSV dengan 15 kolom fitur, mencakup informasi identitas pengguna (*User*, *Card*), nominal transaksi (*Amount*), waktu (*Year*, *Month*, *Day*, *Time*), metode pembayaran (*Use Chip*), identitas *merchant* (*Merchant Name*, *Merchant City*, *Merchant State*, *Zip*, *MCC*), informasi kesalahan (*Errors?*), dan label target (*Is Fraud?*). Rasio kasus penipuan pada dataset ini hanya sekitar 0,12% dari keseluruhan transaksi, menjadikannya representasi nyata dari kondisi *extreme class imbalance*.

Pra-pemrosesan data dilakukan untuk memastikan bahwa data transaksi

dapat digunakan sebagai masukan pada model GraphSAGE. Pada tahap ini, kolom identifier murni seperti *User* dan *Card* tidak digunakan sebagai fitur langsung untuk mencegah kebocoran data (*data leakage*), tetapi tetap dipertahankan sebagai kunci relasional dalam proses konstruksi graf. Nilai yang hilang (*missing values*) pada kolom numerik ditangani menggunakan median imputasi, sedangkan nilai yang hilang pada kolom kategorikal ditangani menggunakan modus imputasi. Setelah itu, kolom *Amount* dinormalisasi menggunakan *min-max scaling* ke rentang $[0, 1]$ agar perbedaan skala tidak mendominasi proses pelatihan model.

Selanjutnya, fitur temporal diekstraksi dari kolom *Year*, *Month*, *Day*, dan *Time* menjadi beberapa fitur turunan, yaitu *hour of day* (0 sampai 23), *day of week* (0 sampai 6), *month* (1 sampai 12), dan *is weekend*. Kolom kategorikal seperti *Use Chip* dan *MCC* dikonversi menggunakan *label encoding*. Sementara itu, informasi lokasi geografis *merchant* yang terdiri dari *Merchant City*, *Merchant State*, dan *Zip* digabungkan dan dikodekan menggunakan *frequency encoding* berdasarkan frekuensi kemunculannya pada data pelatihan. Hasil akhir dari tahap ini adalah matriks fitur transaksi yang siap digunakan dalam proses konstruksi graf.

Setelah proses pra-pemrosesan selesai, dataset dipisahkan menjadi data latih, data validasi, dan data uji. Pemisahan dataset dilakukan secara kronologis berdasarkan urutan waktu transaksi, bukan secara acak murni. Pendekatan ini dipilih agar skenario pengujian lebih mendekati kondisi nyata, di mana model dilatih menggunakan transaksi pada periode sebelumnya dan diuji pada transaksi yang terjadi pada periode setelahnya. Skema pembagian dataset yang digunakan dalam penelitian ini ditunjukkan pada Tabel 3.1.

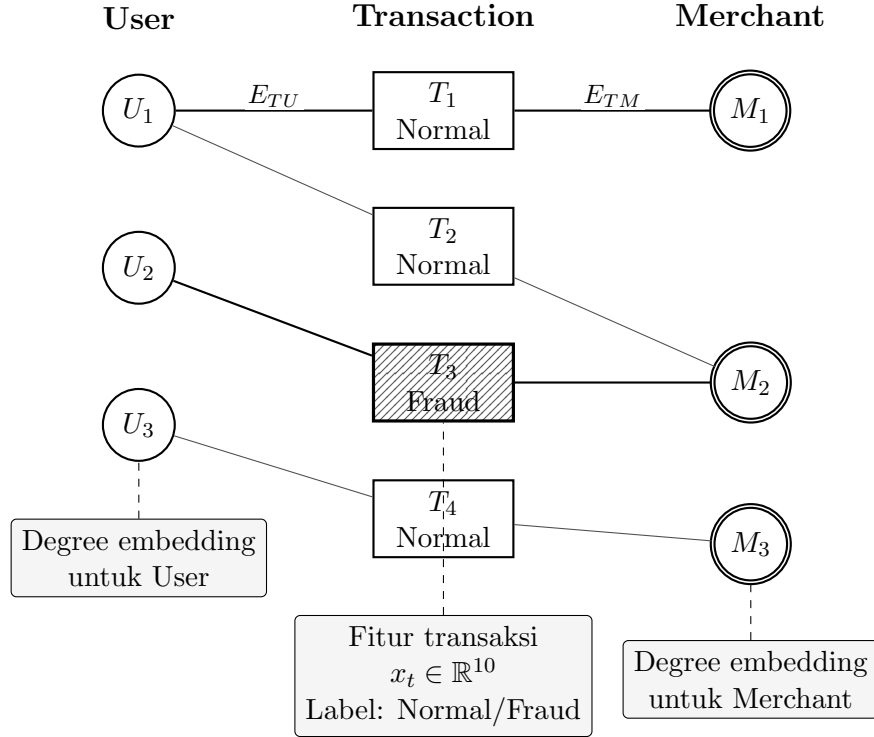
Tabel 3.1: Pembagian dataset penelitian

Subset Data	Proporsi	Fungsi	Keterangan
Data latih (<i>training set</i>)	70%	Digunakan untuk melatih model GraphSAGE pada masing-masing strategi <i>neighbor sampling</i> .	Data ini digunakan untuk mempelajari pola relasional transaksi, memperbarui parameter model, dan menghitung bobot kelas pada fungsi <i>loss</i> .
Data validasi (<i>validation set</i>)	15%	Digunakan untuk memantau performa model selama pelatihan.	Data ini digunakan dalam proses pemilihan konfigurasi terbaik, pemantauan <i>overfitting</i> , dan penerapan <i>early stopping</i> apabila diperlukan.
Data uji (<i>test set</i>)	15%	Digunakan untuk mengevaluasi performa akhir model.	Data ini tidak digunakan selama proses pelatihan dan validasi, sehingga dapat merepresentasikan kemampuan generalisasi model terhadap transaksi baru.

Dengan pembagian tersebut, evaluasi model dapat dilakukan secara lebih adil karena ketiga strategi *neighbor sampling* diuji pada subset data yang sama. Hal ini memastikan bahwa perbedaan performa yang diperoleh benar-benar disebabkan oleh perbedaan mekanisme *sampling*, bukan oleh perbedaan komposisi data yang digunakan pada proses pelatihan dan pengujian.

3.3 Konstruksi Graf Bipartit Transaksi

Representasi data tabular dikonversi ke dalam struktur graf bipartit heterogen $G = (V, E, X)$ dengan tiga tipe node dan dua tipe edge. Himpunan node V terdiri dari node bertipe *User* (V_U) yang diidentifikasi dari kolom *User*, node bertipe *Merchant* (V_M) yang diidentifikasi dari kolom *Merchant Name*, serta node bertipe *Transaction* (V_T) yang merepresentasikan setiap baris transaksi. Himpunan edge E terdiri dari dua tipe yaitu edge yang menghubungkan setiap node transaksi ke node pengguna yang melakukannya (E_{TU}), dan edge yang menghubungkan setiap node transaksi ke node *merchant* tempat transaksi terjadi (E_{TM}). Vektor fitur hasil pra-pemrosesan melekat pada node *Transaction*, sedangkan node *User* dan *Merchant* diberikan representasi awal berupa *degree embedding* satu dimensi yang menyatakan jumlah transaksi yang terhubung. Label biner (fraud atau normal) hanya melekat pada node *Transaction*. Ilustrasi struktur graf bipartit ini disajikan pada Gambar 3.2.



Gambar 3.2: Ilustrasi konstruksi graf bipartit heterogen yang menghubungkan node pengguna, transaksi, dan merchant.

Gambar 3.2 menampilkan diagram node-edge yang menggambarkan struktur graf bipartit dengan tiga tipe node. Node *User* ditampilkan sebagai lingkaran di sisi kiri, node *Transaction* sebagai persegi panjang di bagian tengah, dan node *Merchant* sebagai lingkaran bergaris ganda di sisi kanan. Transaksi normal ditampilkan dengan persegi panjang polos, sedangkan transaksi fraud ditandai dengan arsiran. Setiap node transaksi terhubung ke tepat satu node pengguna melalui edge tipe E_{TU} dan ke tepat satu node merchant melalui edge tipe E_{TM} . Diagram ini juga mengilustrasikan kondisi *extreme class imbalance* dengan menunjukkan jumlah transaksi fraud yang jauh lebih sedikit dibandingkan transaksi normal.

Proses konstruksi graf diimplementasikan menggunakan format **HeteroData** pada pustaka PyTorch Geometric, yang memungkinkan representasi graf heterogen dengan tipe node dan edge yang berbeda. Prosedur konstruksinya dijelaskan secara formal pada Algoritma 1.

3.4 Arsitektur Model GraphSAGE

Arsitektur model GraphSAGE yang digunakan dalam penelitian ini dibuat konsisten untuk seluruh strategi *sampling*. Konsistensi arsitektur diperlukan agar perbedaan performa yang diperoleh dapat dikaitkan dengan mekanisme

Algorithm 1 Prosedur konstruksi graf bipartit heterogen dari dataset CSV.

```

1: procedure KONSTRUKSIGRAF( $D$ )
2:   Input: Dataset CSV  $D$  dengan  $n$  baris transaksi
3:   Output: Graf bipartit heterogen  $G = (V, E, X)$ 
4:   Inisialisasi  $V_U \leftarrow \{\}$ ,  $V_M \leftarrow \{\}$ ,  $V_T \leftarrow \{\}$ ,  $E_{TU} \leftarrow \{\}$ ,  $E_{TM} \leftarrow \{\}$ 
5:   Inisialisasi kamus indeks  $idx\_U$ ,  $idx\_M \leftarrow \{\}$ 
6:    $counter\_U \leftarrow 0$ ,  $counter\_M \leftarrow 0$ 
7:   for setiap baris  $d_i \in D$ ,  $i = 1, \dots, n$  do
8:      $t_i \leftarrow i - 1$  ▷ Indeks node transaksi
9:     if  $d_i[User] \notin idx\_U$  then
10:        $idx\_U[d_i[User]] \leftarrow counter\_U$ 
11:        $counter\_U \leftarrow counter\_U + 1$ 
12:     end if
13:     if  $d_i[Merchant Name] \notin idx\_M$  then
14:        $idx\_M[d_i[Merchant Name]] \leftarrow counter\_M$ 
15:        $counter\_M \leftarrow counter\_M + 1$ 
16:     end if
17:      $u_i \leftarrow idx\_U[d_i[User]]$ 
18:      $m_i \leftarrow idx\_M[d_i[Merchant Name]]$ 
19:     Tambahkan  $(t_i, u_i)$  ke  $E_{TU}$ 
20:     Tambahkan  $(t_i, m_i)$  ke  $E_{TM}$ 
21:     Tambahkan vektor fitur  $\mathbf{x}_i$  ke  $X_T$ 
22:     Tambahkan label  $y_i$  ke  $Y_T$ 
23:   end for
24:   Hitung degree embedding untuk  $V_U$  dan  $V_M$ 
25:   Susun  $G$  sebagai HeteroData dengan  $V_T, V_U, V_M, E_{TU}, E_{TM}, X_T, X_U, X_M, Y_T$ 
26:   Return  $G$ 
27: end procedure

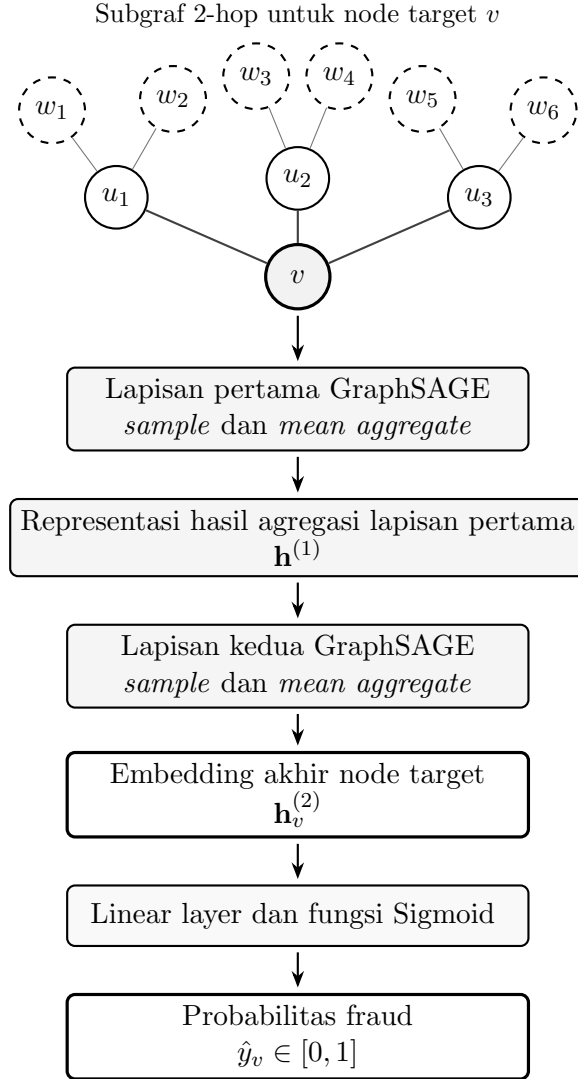
```

sampling, bukan dengan perbedaan kapasitas model. Model terdiri dari dua lapisan GraphSAGE dengan Mean Aggregator [9]. Agregator ini dipilih karena proses pelatihannya stabil dan efisien pada graf berskala besar. Setiap lapisan memiliki dimensi *hidden* sebesar 256 unit, diikuti oleh fungsi aktivasi ReLU dan *Batch Normalization*. Lapisan klasifikasi akhir berupa lapisan linear yang memetakan representasi 256 dimensi ke satu nilai skalar, kemudian ditransformasi menggunakan fungsi Sigmoid untuk menghasilkan probabilitas penipuan dalam rentang $[0, 1]$. Proses *forward pass* untuk node transaksi v pada lapisan ke- l secara formal dinyatakan sebagai berikut.

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \text{MEAN}(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{S}(v)\}) \quad (3.1)$$

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \left[\mathbf{h}_v^{(l-1)} \parallel \mathbf{h}_{\mathcal{N}(v)}^{(l)}\right]\right) \quad (3.2)$$

dengan $\mathbf{h}_v^{(l)}$ menyatakan representasi node v pada lapisan l , $\mathcal{S}(v) \subseteq \mathcal{N}(v)$ menyatakan himpunan tetangga yang disampling dari tetangga v , $\mathbf{W}^{(l)}$ menyatakan matriks bobot yang dapat dilatih, σ menyatakan fungsi aktivasi ReLU, dan $\parallel$ menyatakan operasi konkatenasi vektor. Ilustrasi arsitektur dua lapisan ini disajikan pada Gambar 3.3.



Gambar 3.3: Ilustrasi arsitektur GraphSAGE dua lapisan pada subgraf 2-hop

Gambar 3.3 menunjukkan alur pembentukan representasi node pada arsitektur GraphSAGE dua lapisan. Node target v membentuk subgraf 2-hop melalui proses pengambilan sampel tetangga. Pada lapisan pertama, informasi dari tetangga 2-hop diintegrasikan untuk membentuk representasi tetangga 1-hop. Pada lapisan kedua, representasi tetangga 1-hop diintegrasikan menuju node target sehingga diperoleh embedding akhir $\mathbf{h}_v^{(2)}$. Embedding tersebut kemudian diproses oleh lapisan linear dan fungsi Sigmoid untuk menghasilkan probabilitas bahwa transaksi termasuk ke dalam kelas fraud.

3.5 Perancangan Strategi *Neighbor Sampling*

Seluruh strategi *neighbor sampling* yang dirancang dalam penelitian ini diimplementasikan sebagai modifikasi pada fungsi `NeighborSampler` di pustaka PyTorch Geometric. Setiap strategi menggunakan ukuran sampel tetangga yang sama, yaitu $k = 25$ pada lapisan pertama dan $k = 10$ pada lapisan kedua, mengikuti rekomendasi dari Hamilton et al. [9]. Dengan pengaturan ini, perbedaan antar strategi hanya terletak pada distribusi probabilitas pemilihan tetangga.

3.5.1 *Uniform Random Sampling* (Baseline)

Uniform Random Sampling merupakan mekanisme *sampling* bawaan GraphSAGE yang memilih k tetangga secara acak dari himpunan tetangga node target menggunakan distribusi seragam tanpa penggantian (*without replacement*). Seluruh tetangga memiliki probabilitas pemilihan yang sama, yaitu $1/|\mathcal{N}(v)|$, tanpa mempertimbangkan relevansi topologis maupun distribusi label. Strategi ini ditetapkan sebagai *baseline* untuk mengukur peningkatan performa yang dapat diberikan oleh strategi yang dioptimalkan. Prosedurnya dinyatakan secara formal pada Algoritma 2.

Algorithm 2 Prosedur *Uniform Random Sampling* pada GraphSAGE [9].

```

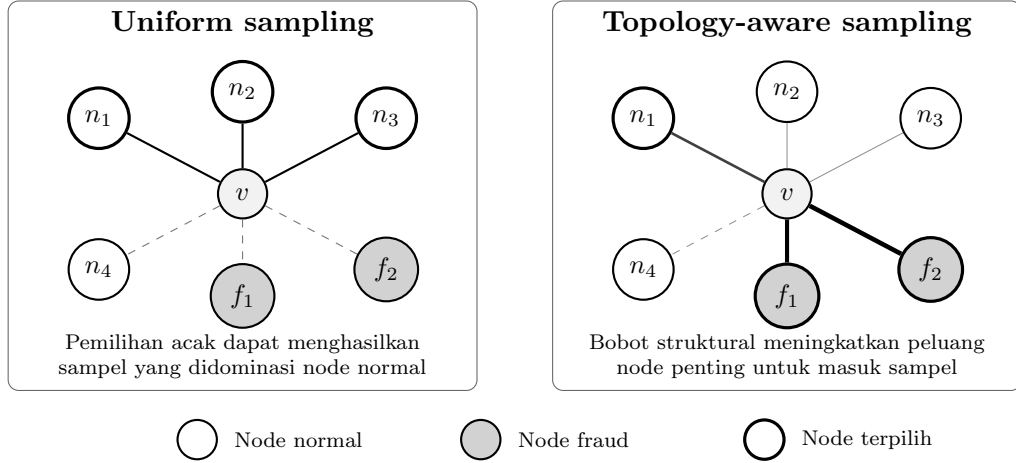
1: procedure UNIFORMSAMPLING( $v, \mathcal{N}(v), k$ )
2:   Input: Node target  $v$ , himpunan tetangga  $\mathcal{N}(v)$ , ukuran sampel  $k$ 
3:   Output: Himpunan tetangga yang disampling  $\mathcal{S}(v)$ 
4:   if  $|\mathcal{N}(v)| \leq k$  then
5:     Return  $\mathcal{N}(v)$     ▷ Ambil semua jika tetangga lebih sedikit dari  $k$ 
6:   end if
7:   Tetapkan  $p(u) \leftarrow \frac{1}{|\mathcal{N}(v)|}$  untuk setiap  $u \in \mathcal{N}(v)$ 
8:    $\mathcal{S}(v) \leftarrow$  sampel  $k$  node dari  $\mathcal{N}(v)$  menggunakan distribusi  $p$  tanpa
   penggantian
9:   Return  $\mathcal{S}(v)$ 
10: end procedure

```

3.5.2 *Topology-Aware Sampling*

Topology-Aware Sampling dirancang untuk mengatasi kelemahan *uniform sampling* dalam kondisi *heterophily* tinggi. Strategi ini menetapkan bobot pemilihan yang berbeda-beda untuk setiap tetangga kandidat berdasarkan dua ukuran kemiripan struktural. Ukuran pertama adalah *Jaccard Similarity* antara himpunan tetangga node target v dan himpunan tetangga kandidat u , yang mengukur sejauh mana kedua node berbagi lingkungan topologis yang sama. Ukuran kedua adalah *Local Homophily Ratio* yang mengukur proporsi tetangga node kandidat u yang memiliki label sama dengan node target v

berdasarkan riwayat label pada data pelatihan. Bobot akhir untuk setiap kandidat tetangga merupakan rata-rata tertimbang dari kedua ukuran tersebut, kemudian dinormalisasi menjadi distribusi probabilitas. Ilustrasi perbedaan antara *uniform sampling* dan *topology-aware sampling* pada kasus nyata disajikan pada Gambar 3.4.



Gambar 3.4: Perbandingan uniform sampling dan topology-aware sampling pada pemilihan tetangga node target

Gambar 3.4 memperlihatkan perbedaan mekanisme pemilihan tetangga antara *uniform sampling* dan *topology-aware sampling*. Pada *uniform sampling*, setiap tetangga memiliki peluang yang sama untuk dipilih tanpa mempertimbangkan posisi atau kontribusinya dalam struktur graf. Kondisi tersebut dapat menyebabkan sampel didominasi oleh node normal, sehingga informasi dari node fraud yang jumlahnya lebih sedikit menjadi kurang terwakili. Sebaliknya, *topology-aware sampling* mempertimbangkan informasi struktural graf, seperti kekuatan hubungan, kedekatan terhadap node target, dan skor kepentingan tetangga. Dengan mekanisme ini, node yang lebih relevan terhadap identifikasi pola fraud memiliki peluang lebih tinggi untuk masuk ke dalam sampel.

3.5.3 Importance-Based Sampling

Importance-Based Sampling dirancang untuk memprioritaskan tetangga yang paling informatif berdasarkan tingkat kepentingannya dalam struktur graf global maupun lokal. Berbeda dari *Topology-Aware Sampling* yang berfokus pada kemiripan struktural, strategi ini mengukur seberapa besar pengaruh suatu kandidat tetangga terhadap distribusi informasi dalam subgraf lokal. Ukuran kepentingan yang dikombinasikan adalah *Degree Centrality* yang dinormalisasi sebagai $d_u/(|V| - 1)$ dan skor *Personalized PageRank* (PPR) yang dihitung secara lokal menggunakan propagasi terbatas r langkah dari node target untuk menjaga efisiensi komputasi pada graf berskala besar. Node dengan

Algorithm 3 Prosedur *Topology-Aware Sampling* untuk GraphSAGE (daptasi dari konsep pada [12] dan [15]).

```

1: procedure TOPOLOGYAWARESAMPLING( $v, \mathcal{N}(v), k, L, G$ )
2:   Input: Node target  $v$ , himpunan tetangga  $\mathcal{N}(v)$ , ukuran sampel  $k$ ,
      riwayat label  $L$ , struktur graf  $G$ 
3:   Output: Himpunan tetangga yang disampling  $\mathcal{S}(v)$ 
4:   if  $|\mathcal{N}(v)| \leq k$  then
5:     Return  $\mathcal{N}(v)$ 
6:   end if
7:   for setiap  $u \in \mathcal{N}(v)$  do
8:     Hitung Jaccard Similarity:

```

$$J(v, u) = \frac{|\mathcal{N}(v) \cap \mathcal{N}(u)|}{|\mathcal{N}(v) \cup \mathcal{N}(u)|}$$

```

9:     Hitung Local Homophily Ratio:

```

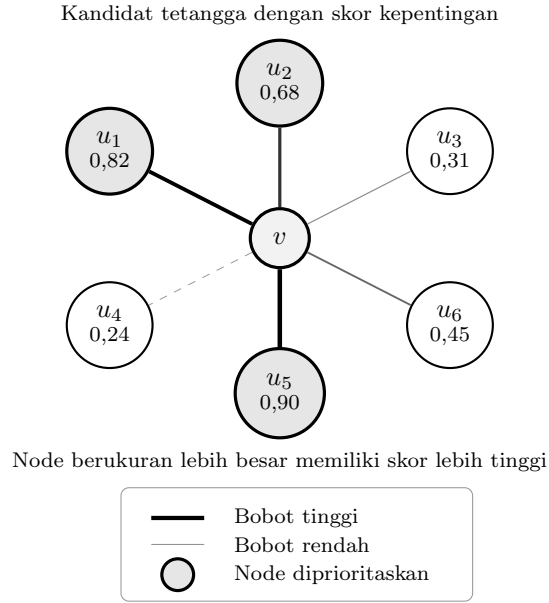
$$H(v, u) = \frac{|\{w \in \mathcal{N}(u) : L[w] = L[v]\}|}{|\mathcal{N}(u)| + 1}$$

```

10:    Hitung bobot gabungan:  $w(v, u) \leftarrow \alpha \cdot J(v, u) + (1 - \alpha) \cdot H(v, u)$ 
      dengan  $\alpha = 0,5$ 
11:  end for
12:  Normalisasi:  $p(u) \leftarrow w(v, u) / \sum_{u' \in \mathcal{N}(v)} w(v, u')$  untuk setiap  $u \in \mathcal{N}(v)$ 
13:   $\mathcal{S}(v) \leftarrow$  sampel  $k$  node dari  $\mathcal{N}(v)$  menggunakan distribusi  $p$  tanpa
      penggantian
14:  Return  $\mathcal{S}(v)$ 
15: end procedure

```

skor PPR tinggi terhadap node target berarti memiliki hubungan topologis yang kuat dan karenanya menyumbang informasi agregasi yang lebih relevan. Ilustrasi konsep skor kepentingan node ini ditampilkan pada Gambar 3.5.



Gambar 3.5: Ilustrasi skor kepentingan node pada strategi *Importance-Based Sampling*

Gambar 3.5 menunjukkan ilustrasi perhitungan skor kepentingan pada kandidat tetangga node target. Node target v ditempatkan di bagian tengah, sedangkan kandidat tetangga berada di sekelilingnya. Ukuran lingkaran merepresentasikan besar kecilnya skor kepentingan gabungan yang diperoleh dari *Degree Centrality* dan *Personalized PageRank*. Semakin besar ukuran node, semakin tinggi prioritas node tersebut untuk dipilih sebagai sampel. Ketebalan edge menunjukkan bobot hubungan antara node target dan kandidat tetangganya. Edge yang lebih tebal menunjukkan hubungan yang lebih kuat, sedangkan edge yang lebih tipis menunjukkan hubungan yang lebih lemah. Dengan pendekatan ini, *Importance-Based Sampling* tidak memilih tetangga secara acak, tetapi memprioritaskan node yang memiliki kontribusi informasi lebih besar terhadap proses agregasi GraphSAGE.

3.6 Prosedur Pelatihan Model

Prosedur pelatihan yang sama diterapkan pada seluruh strategi *sampling*. Model dilatih menggunakan *mini-batch training* dengan ukuran *batch* sebesar 1.024 node transaksi per iterasi. Fungsi *loss* yang digunakan adalah *Binary Cross-Entropy* dengan pembobotan kelas (*class-weighted BCE*) menggunakan rasio invers frekuensi, sehingga kelas fraud yang minoritas mendapat bobot

Algorithm 4 Prosedur *Importance-Based Sampling* untuk GraphSAGE (daptasi dari konsep *Personalized PageRank* pada Andersen et al. (2006) yang disesuaikan untuk *mini-batch* GraphSAGE).

```

1: procedure IMPORTANCEBASEDSAMPLING( $v, \mathcal{N}(v), k, G, r$ )
2:   Input: Node target  $v$ , himpunan tetangga  $\mathcal{N}(v)$ , ukuran sampel  $k$ ,
   struktur graf  $G$ , jumlah langkah propagasi PPR  $r$ 
3:   Output: Himpunan tetangga yang disampling  $\mathcal{S}(v)$ 
4:   if  $|\mathcal{N}(v)| \leq k$  then
5:     Return  $\mathcal{N}(v)$ 
6:   end if
7:   Inisialisasi vektor PPR:  $\pi_v \leftarrow \mathbf{0}, \pi_v[v] \leftarrow 1$ 
8:   for langkah  $t = 1, \dots, r$  do  $\triangleright$  Propagasi PPR lokal terbatas  $r$  langkah
9:      $\pi_v \leftarrow (1 - \beta) \cdot A_{\text{norm}} \cdot \pi_v + \beta \cdot \mathbf{e}_v$   $\triangleright \beta = 0,15$ 
    adalah faktor teleportasi,  $A_{\text{norm}}$  adalah matriks adjacency ternormalisasi,
     $\mathbf{e}_v$  adalah vektor basis untuk node  $v$ 
10:  end for
11:  for setiap  $u \in \mathcal{N}(v)$  do
12:    Hitung Degree Centrality:  $C_d(u) \leftarrow |\mathcal{N}(u)| / (|V| - 1)$ 
13:    Ambil skor PPR:  $C_p(u) \leftarrow \pi_v[u]$ 
14:    Hitung bobot gabungan:  $w(v, u) \leftarrow \gamma \cdot C_d(u) + (1 - \gamma) \cdot C_p(u)$  dengan
     $\gamma = 0,3$ 
15:  end for
16:  Normalisasi:  $p(u) \leftarrow w(v, u) / \sum_{u' \in \mathcal{N}(v)} w(v, u')$  untuk setiap  $u \in \mathcal{N}(v)$ 
17:   $\mathcal{S}(v) \leftarrow$  sampel  $k$  node dari  $\mathcal{N}(v)$  menggunakan distribusi  $p$  tanpa
  penggantian
18:  Return  $\mathcal{S}(v)$ 
19: end procedure

```

sekitar $1/0,0012 \approx 833$ kali lebih besar dibandingkan kelas normal. Optimizer yang digunakan adalah Adam dengan *learning rate* awal sebesar 0,001 dan penurunan *learning rate* sebesar faktor 0,5 apabila nilai AUPRC pada data validasi tidak meningkat selama lima *epoch* berturut-turut. Pelatihan dihentikan menggunakan mekanisme *early stopping* berbasis AUPRC validasi dengan *patience* sebesar 15 *epoch*. Model dengan nilai AUPRC validasi terbaik disimpan sebagai model final. Seluruh eksperimen dijalankan sebanyak lima kali dengan *random seed* yang berbeda untuk memastikan kestabilan hasil. Prosedur lengkap pelatihan dinyatakan pada Algoritma 5.

3.7 Skenario Pengujian dan Evaluasi

Setelah pelatihan selesai, setiap model dievaluasi melalui skenario yang saling melengkapi. Skenario performa keseluruhan digunakan untuk membandingkan F1-Score, *Recall*, dan AUPRC dari seluruh model pada data uji. Skenario deteksi *long-tail fraud* digunakan untuk mengukur ketahanan model terhadap transaksi fraud yang tersamar di antara mayoritas tetangga normal. Pada skenario ini, subset data uji dibatasi pada transaksi penipuan yang terhubung dengan lebih dari 70% tetangga bernilai normal sebagai representasi kondisi kamufase tinggi. Skenario efisiensi inferensi digunakan untuk mengukur *inference time* per 1.000 node dalam mode evaluasi tanpa gradien. Nilai latensi tersebut kemudian dibandingkan dengan peningkatan AUPRC agar kelayakan penerapan pada sistem *real-time* dapat dianalisis. Protokol evaluasi keseluruhan diilustrasikan pada Gambar 3.6.

Algorithm 5 Prosedur pelatihan model GraphSAGE dengan *custom sampler* dan *early stopping* berbasis AUPRC (diadaptasi dari [9]).

```

1: procedure PELATIHANGRAPHSAGE( $G, M, \text{Sampler}, E_{\max}, B, \text{patience}$ )
2:   Input: Graf  $G$ , model GraphSAGE  $M$ , fungsi sampler  $\text{Sampler}$  (salah
      satu dari Algoritma 2, 3, atau 4), epoch maksimum  $E_{\max}$ , ukuran batch  $B$ ,
      patience
3:   Output: Model terlatih  $M^*$ 
4:   Bagi  $G$  menjadi  $G_{\text{train}}, G_{\text{val}}, G_{\text{test}}$  dengan rasio 80:10:10 (pemisahan
      temporal)
5:   Inisialisasi  $\text{best\_auprc} \leftarrow 0, \text{no\_improve} \leftarrow 0, M^* \leftarrow M$ 
6:   for  $e = 1, \dots, E_{\max}$  do
7:     Acak urutan node transaksi pada  $G_{\text{train}}$ 
8:     for setiap batch  $\mathcal{B}$  berukuran  $B$  dari  $G_{\text{train}}$  do
9:       Bangun subgraph dengan memanggil Sampler untuk setiap node
      dalam  $\mathcal{B}$ 
10:      Hitung forward pass:  $\hat{Y} \leftarrow M(\mathcal{B}_{\text{subgraph}})$ 
11:      Hitung loss:  $\mathcal{L} \leftarrow \text{WeightedBCE}(\hat{Y}, Y_{\mathcal{B}})$ 
12:      Lakukan backward pass dan perbarui parameter  $M$  menggunak-
      an Adam
13:     end for
14:     Hitung AUPRC pada  $G_{\text{val}}$  tanpa gradien
15:     if  $\text{AUPRC}_{\text{val}} > \text{best\_auprc}$  then
16:        $\text{best\_auprc} \leftarrow \text{AUPRC}_{\text{val}}$ 
17:        $M^* \leftarrow$  salin parameter  $M$  saat ini
18:        $\text{no\_improve} \leftarrow 0$ 
19:     else
20:        $\text{no\_improve} \leftarrow \text{no\_improve} + 1$ 
21:     end if
22:     if  $\text{no\_improve} \geq \text{patience}$  then
23:       Break  $\triangleright$  Early stopping
24:     end if
25:   end for
26:   Return  $M^*$ 
27: end procedure

```



Gambar 3.6: Bagan alur protokol evaluasi komparatif strategi *neighbor sampling*

Gambar 3.6 menunjukkan alur evaluasi komparatif terhadap seluruh varian model yang telah dilatih. Proses evaluasi dimulai dari model terlatih dan data uji yang belum pernah digunakan pada tahap pelatihan. Keduanya digunakan pada tahap *forward pass* dengan pencatatan waktu inferensi. Hasil dari proses tersebut berupa skor probabilitas fraud yang kemudian dikonversi menjadi label biner menggunakan *threshold* 0,5. Label prediksi dan label aktual selanjutnya digunakan untuk membangun *confusion matrix*.

Berdasarkan hasil tersebut, evaluasi diarahkan pada performa keseluruhan, kemampuan deteksi *long-tail fraud*, dan efisiensi inferensi. Performa keseluruhan diukur menggunakan F1-Score, *Recall*, dan AUPRC. Kemampuan deteksi *long-tail fraud* dianalisis untuk menilai kemampuan model dalam mendeteksi

transaksi fraud yang tersamar di antara mayoritas tetangga normal. Efisiensi inferensi dianalisis untuk melihat kebutuhan waktu komputasi dari masing-masing strategi *sampling*. Seluruh hasil kemudian dirangkum dalam tabel komparatif agar dapat dianalisis secara berdampingan.

Implementasi eksperimen direncanakan menggunakan bahasa pemrograman Python dengan pustaka pembelajaran mendalam yang mendukung pemodelan graf. Versi pustaka, spesifikasi perangkat komputasi, dan konfigurasi eksperimen akan dicatat secara rinci pada tahap pelaksanaan penelitian. Setiap skenario pengujian dirancang untuk dilakukan lebih dari satu kali agar hasil evaluasi dapat dilaporkan dalam bentuk nilai rata-rata dan standar deviasi.

Daftar Pustaka

- [1] Yao Zou and Dawei Cheng. Effective high-order graph representation learning for credit card fraud detection. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, pages 7581–7589. IJCAI Organization, 2024.
- [2] Chaoli Lou, Yueyang Wang, Jianing Li, Yueru Qian, and Xiuhua Li. Graph neural network for fraud detection via context encoding and adaptive aggregation. *Expert Systems with Applications*, 261:125473, 2025.
- [3] Fawaz K. Alarfaj, Iqra Malik, Hikmat Ullah Khan, Naif Almusallam, Muhammad Ramzan, and Muneer Ahmed. Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms. *IEEE Access*, 10:39700–39715, 2022.
- [4] Soroor Motie and Bijan Raahemi. Financial fraud detection using graph neural networks: A systematic review. *Expert Systems with Applications*, 240:122156, 2024.
- [5] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S. Yu. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, pages 315–324. ACM, 2020.
- [6] Rafaël Van Belle, Bart Baesens, and Jochen De Weerd. CATCHM: A novel network-based credit card fraud detection method using node representation learning. *Decision Support Systems*, 164:113866, 2023.
- [7] Sheng Xiang, Mingqi Zhu, Dawei Cheng, Enxia Li, Ruihui Zhao, Yi Ouyang, Ling Chen, and Yefeng Zheng. Semi-supervised credit card fraud detection via attribute-driven graph representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 14557–14565, 2023.
- [8] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. GraphSAINT: Graph sampling based inductive

- learning method. In *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*, 2020.
- [9] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NIPS 2017)*, volume 30, 2017.
 - [10] Rafaël Van Belle, Charles Van Damme, Hendrik Tytgat, and Jochen De Weerd. Inductive graph representation learning for fraud detection. *Expert Systems with Applications*, 193:116463, 2022.
 - [11] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. Pick and choose: A GNN-based imbalanced learning approach for fraud detection. In *Proceedings of the Web Conference 2021 (WWW ’21)*, pages 3168–3177. ACM, 2021.
 - [12] Mallika Boyapati and Ramazan S. Aygun. BalancerGNN: Balancer graph neural networks for imbalanced datasets: A case study on fraud detection. *Neural Networks*, 182:106926, 2025.
 - [13] Fengzhao Shi, Yanan Cao, Yanmin Shang, Yuchen Zhou, Chuan Zhou, and Jia Wu. H2-FDetector: A GNN-based fraud detector with homophilic and heterophilic connections. In *Proceedings of the ACM Web Conference 2022 (WWW ’22)*, pages 1486–1494. ACM, 2022.
 - [14] Sharon Altman. IBM synthetic credit card transactions dataset. Kaggle, 2019.
 - [15] Jing Chen, Quanzhen Chen, Feng Jiang, Xuyao Guo, Kaiyue Sha, and Yuxuan Wang. SCN_GNN: A GNN-based fraud detection algorithm combining strong node and graph topology information. *Expert Systems with Applications*, 237:121643, 2024.
 - [16] Xuanwen Huang, Yang Yang, Yang Wang, Chunping Wang, Zhisheng Zhang, Jiarong Xu, Lei Chen, and Michalis Vazirgiannis. DGraph: A large-scale financial dataset for graph anomaly detection. In *Advances in Neural Information Processing Systems (NeurIPS 2022) Track on Datasets and Benchmarks*, 2022.
 - [17] Bin Wu, Xinyu Yao, Boyan Zhang, Kuo-Ming Chao, and Yinsheng Li. SplitGNN: Spectral graph neural network for fraud detection against heterophily. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM ’23)*. ACM, 2023.

- [18] Guoxiang Tong and Jieyu Shen. Financial transaction fraud detector based on imbalance learning and graph neural network. *Applied Soft Computing*, 149:110984, 2023.
- [19] Yue Tian, Guanjin Liu, Jiacun Wang, and Mengchu Zhou. ASA-GNN: Adaptive sampling and aggregation-based graph neural network for transaction fraud detection. *IEEE Transactions on Computational Social Systems*, 11(3):3536–3549, 2024.
- [20] Qiutong Li, Yanshen He, Cong Xu, Feng Wu, Jianliang Gao, and Zhao Li. Dual-augment graph neural network for fraud detection. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22)*, pages 4545–4549. ACM, 2022.
- [21] Mengda Huang, Yang Liu, Xiang Ao, Kuan Li, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. AUC-oriented graph neural network for fraud detection. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*, pages 1311–1321. ACM, 2022.

Lampiran